

Universität Leipzig
Institut für Informatik

Untersuchungen zu einer hierarchischen Variante der
ACO-Metaheuristik zur Funktionsoptimierung

Masterarbeit

Leipzig, Januar, 2025

vorgelegt von

Felix Cäsar Madorskiy
Studiengang Informatik M.Sc.

Betreuender Hochschullehrer: Prof. Dr. Martin Middendorf
Fakultät für Mathematik und Informatik/Institut für Informatik/
Schwarmintelligenz und Komplexe Systeme

Inhaltsverzeichnis

Abkürzungsverzeichnis

1. Einleitung	1
2. Grundlagen	2
2.1. Einführung in Ameisenalgorithmen	2
2.1.1. Biologische Beschreibung der Nahrungssuche von Ameisen	2
2.1.2. Mathematische Beschreibung der Nahrungssuche von Ameisen	2
2.1.3. Permutationsprobleme	4
2.1.4. Traveling Salesperson Problem (TSP)	5
2.2. ACO-Metaheuristik nach <i>Dorigo et al.</i> [7]	5
2.3. Weiterentwicklung der ACO Metaheuristik zu der P-ACO-Metaheuristik	6
2.4. Ameisenalgorithmen Mit Einem Stetigen Lösungsarchiv	8
2.5. Ameisenalgorithmen Mit Einem Gemischten Lösungsarchiv	8
2.6. Ameisenalgorithmen mit einem Hierarchischen Gemischten Lösungsarchiv	9
2.7. Ameisenalgorithmen in stetigen Variablen	9
3. Optimierungsprobleme	10
4. Grundlagen	10
4.1. $ACO_{\mathbb{R}}$	11
4.2. $ACO_{\mathbb{R}}$ -simple	12
4.3. $ACO_{\mathbb{R}}$ -very-simple	13
4.4. Erweiterung von $ACO_{\mathbb{R}}$ -very-simple für Probleme mit nicht-stetigen Variablen	14
5. SPPBO-Schema	15
6. HSPPBO-Schema	16
7. Formulierung Von $ACO_{\mathbb{R}}$-VS Im HSPPBO-Schema	18
8. Kombinieren des HSPPBO-Schemas mit $ACO_{\mathbb{R}}$-very-simple	18
8.1. Hom-HPB- $ACO_{\mathbb{R}}$ -very-simple	18
8.2. Het-HPB- $ACO_{\mathbb{R}}$ -very-simple	18
8.3. Het-HSPPB- $ACO_{\mathbb{R}}$ -very-simple	19
9. Testfunktionen	19
9.1. Permutationsprobleme Testfunktionen	21
9.1.1. Traveling Salesperson Problem (TSP)	21
9.1.2. Quadratic Assignment Problem (QAP)	21
9.2. Stetige Testfunktionen	21
9.2.1. Testfunktion - Paraboloid	27

9.2.2. Testfunktion - Easom	28
9.2.3. Testfunktion - Griewangk	29
9.2.4. Testfunktion - Rosenbrock	29
9.2.5. Testfunktion - Hartmann	30
9.2.6. Testfunktion - Shekel	31
9.3. Diskrete Testfunktionen	31
9.3.1. Ordinale Testfunktionen mit gleichverteiltem Definitionsbereich .	31
9.3.2. Ordinale Testfunktionen mit asymmetrischem Definitionsbereich .	31
9.4. Testfunktionen mit gemischten Variablen	32
9.4.1. Coil Spring Design Problem (CSD)	32
9.4.2. Thermal Insulation System Design Problem (TISD)	37
9.4.3. Traveling Spice Salesperson Problem (TSSP)	37
10. Herausforderung	37
11. Einfluss der Parameter	37
11.1. Einfluss der Anzahl der Ameisen	38
11.2. Einfluss des Parameters q	38
11.3. Einfluss des Parameters k	39
11.4. Einfluss des Parameters ξ	40
11.5. Auswertung der Einflüsse der Parameter	40
12. Vergleich zweier Intervallberechnungen für $ACO_{\mathbb{R}}$-simple und $ACO_{\mathbb{R}}$-very-simple	41
13. Vergleich von $ACO_{\mathbb{R}}$, $ACO_{\mathbb{R}}$-simple und $ACO_{\mathbb{R}}$-very-simple	43
14. Konvergenzbetrachtung	50
14.1. Rosenbrock	51
15. Kurzzusammenfassung	52
Literaturverzeichnis	53
Tabellenverzeichnis	55
Abbildungsverzeichnis	55
Anlagen	I
A. Einfluss der Anzahl der Ameisen	I
B. Einfluss des Parameters q	II
C. Einfluss des Parameters ξ	III

Abkürzungsverzeichnis

ACO_ℝ-VS	ACO _ℝ -very-simple
CSD Problem	Coil Spring Design Problem
SCE	solution creating entities
TSP	Traveling Salesperson Problem

1. Einleitung

Die Informatik als Wissenschaft gliedert sich in viele Bereiche, eines davon ist das Entwickeln von Algorithmen zum heuristischen Berechnen von Näherungslösungen von Problemen, von denen ausgegangen wird, dass es für sie keine effizienten Algorithmen gibt und somit eine Näherungslösung das Beste ist, was erreicht werden kann als Kompromiss. Ferner gibt es keine Möglichkeit zur Verifizieren, ob diese so gefundenen Lösungen tatsächlich Lösungen zu den Problemen sind, sie sind nur gut genug um in der Praxis stellvertretend als angenäherte Lösungen zu dienen.

Die, für das heuristische Berechnen von Lösungen, verwendeten Algorithmen können weiter aufgeteilt werden in Klassen von Algorithmen. Die hier relevante Klasse sind Ameisenalgorithmen, um genau zu sein, Ameisenalgorithmen in stetigen Variablen. Diese Klassen können weiter aufgeteilt werden in Varianten zur Lösungserzeugung. Konkret beschäftigt sich diese Arbeit mit dem Vergleich zweier Varianten der Lösungserzeugung von Ameisenalgorithmen, die erste Variante repräsentiert durch den Algorithmus $ACO_{\mathbb{R}}$ und die andere durch die Algorithmen $ACO_{\mathbb{R}}$ -simple und $ACO_{\mathbb{R}}$ -very-simple.

Ameisenalgorithmen begannen ihre Entwicklung durch die Inspiration von dem Verhalten von Ameisen bei der Nahrungssuche. Dadurch waren die ersten Ameisenalgorithmen dafür gedacht, Anordnungsprobleme, Permutationsprobleme, zu lösen, worin sie sehr erfolgreich waren, z.B. bei der Routenplanung für Fahrzeuge. Sie hatten nur ein Nachteil, sie waren nur für diskrete Variablen einsetzbar, also Variablen, die nur endlich viele Werte annehmen konnten. Dies limitierte jedoch den Einsatzbereich der Algorithmen für Probleme, wie sie in der Praxis oft vorkommen, Probleme in stetigen Variablen, also Probleme mit Variablen, die beliebige Werte annehmen können.

Dieses Hindernis wurde jedoch bereits überwunden, durch gleich mehrere Varianten der Lösungserzeugung von Ameisenalgorithmen, u.a. Continuous ACO (CACO) vorgestellt in *Bilchev and Parmee* [1] und dem hier betrachteten Algorithmus $ACO_{\mathbb{R}}$ vorgestellt in *Socha und Dorigo* [22]. Doch alle Varianten der Lösungserzeugung in stetigen Variablen, welche durch Ameisenalgorithmen inspiriert wurden, zeichnen sich dadurch aus, dass ihre Verwendung von bereits gefunden Lösungen und das Zusammensetzen dieser zu neuen Lösungen eine gewisse Komplexität aufweist.

Genau mit diesem Problem beschäftigt sich die hier vorliegende Arbeit durch das Vorstellen einer sehr stark vereinfachten Variante zur Lösungserzeugung eines Ameisenalgorithmus und dem Vergleich dieser Variante mit der, wie das Paper von *Socha und Dorigo* [22] selbst aussagt, ACO im Geiste treuesten Variante, $ACO_{\mathbb{R}}$. Wir beantworten also in dieser Arbeit die Frage, durch den Vergleich der beiden Varianten, ob die Komplexität wirklich nötig ist.

2. Grundlagen

2.1. Einführung in Ameisenalgorithmen

Wie in meiner Bachelorarbeit [16] beginnen wir zunächst mit der biologischen und mathematischen Beschreibung der Nahrungssuche von Ameisen.

2.1.1. Biologische Beschreibung der Nahrungssuche von Ameisen

Wie *Bonabeau, Dorigo und Theraulaz* [3] und *Blum und Merkle* [6] beschrieben haben, hinterlassen Ameisen, bei der Nahrungssuche, wenn sie sich zwischen Nest und Futter bewegen, eine Pheromonspur. Die Menge des hinterlassenen Pheromons hängt von der Quantität und Qualität des gefundenen Futters ab. Wenn Ameisen Nest oder Futter verlassen, haben die Wege auf denen Pheromon liegt, eine höhere Wahrscheinlichkeit wieder begangen zu werden, je höher die Menge des hinterlassenen Pheromons auf einem Weg, desto höher die Wahrscheinlichkeit, dass der Weg von Ameisen begangen wird. Wenn eine Ameise also den optimalen, also kürzesten Weg zwischen Nest und Futter gefunden hat, aggregiert sich Pheromon auf diesem Weg schneller, da Ameisen, die sich für den kürzesten Weg entschieden haben, da der Weg der kürzeste ist, vom Futter schon zurückkommen, während andere Ameisen noch unterwegs sind. Zusätzlich verdunstet Pheromon über Zeit, was dazu führt, dass seltener begangene Wege zunehmend weniger Pheromon aufweisen und damit unwahrscheinlicher begangen werden. Diese beiden Mechanismen zusammen sorgen dafür, dass sich die kürzesten Wege über Zeit herauskristallisieren.

2.1.2. Mathematische Beschreibung der Nahrungssuche von Ameisen

In diesem Abschnitt betrachten wir ein ideales Futter, also ein Futter, dass in Quantität und Qualität konstant bleibt, und ideale Ameisen, also Ameisen, die mit konstanter Geschwindigkeit sich geradlinig bewegen. Wir müssen diese Annahmen treffen, da, wie im Abschnitt 2.1.1 beschrieben, die Menge des auf einem Weg hinterlassenen Pheromons von der Quantität und Qualität des Futters abhängt und wie in *Camazine et al.* [4] beschrieben, reale Ameisen auf einem Weg umdrehen können und wieder zurücklaufen von woher sie kamen. Ferner, in Anlehnung an *Dorigo und Stützle* [8] und *Engelbrecht* [9], betrachten wir eine ideale Experimentaufstellung in Form eines Tupels $(\Sigma, \mathcal{E}, \Delta, \mathcal{M}, f, \mathcal{I}, \mathcal{S})$.

Dabei ist Σ der Suchraum, welcher hier der ungerichtete Graph $G(V, E)$ ist, der aus drei Orten/Knoten, an denen sich Ameisen aufhalten können und drei Wegen zwischen diesen Orten, besteht. Der Graph befindet sich in der euklidischen Ebene. Die Orte sind Nest (N), Food (F) und Umweg/Detour (D), also gilt $V = \{N, F, D\}$. Die Wege verbinden die Orte, wie Abbildung 1 zu sehen. Die Wege haben alle die Länge 1 Längeneinheit (LE) und können in beide Richtungen begangen werden, also gilt $E = \{\{NF\}, \{ND\}, \{DF\}\}$. Zusammengefasst gilt $G = (\{N, F, D\}, \{\{NF\}, \{ND\}, \{DF\}\})$. Die Menge der idealen Ameisen bzw. der solution creating entities (SCE) ist $\mathcal{E}$. Hier gilt $|\mathcal{E}| = 1$, es wird also immer wieder eine ideale Ameise in diesem Experiment betrachtet.

Die Menge des Pheromons, welches die Ameise auf die von ihr gewählten Wege hinterlässt, ist Δ . In unserem Experiment, angelehnt an [3] und [6], gilt $\Delta = 1$. Die Menge des Pheromons ρ auf allen Wegen, also die Pheromoninformation, wird in der Pheromonmatrix $\mathcal{M}$ gespeichert. Dabei steht jeder Matrixeintrag für eine Kante im Graph G und der Wert des Eintrages steht für die Menge von Pheromon, welches auf der Kante liegt. Die Zielfunktion ist f . Die Zielfunktion gibt die Summe der Weglängen der Wege an, zwischen N und F, welche die Ameisen gegangen sind. In unserer idealen Experimentaufstellung gilt also $f(x) \in \{1, 2\}$, da die Ameisen entweder den direkten Weg $\{NF\}$ gehen oder den Umweg $\{ND, DF\}$ gehen kann und damit gilt $f(\{NF\}) = 1$ oder $f(\{ND, DF\}) = 2$. Das Begehen eines Weges verbraucht exakt eine Zeiteinheit, also verbraucht der direkte Weg $\{NF\}$ eine Zeiteinheit und der Weg über den Umweg $\{ND, DF\}$ Zwei. Das Ergebnis des in diesem Abschnitt beschriebenen Vorgehens, und das Ziel für zukünftige Problemstellungen, ist das finden eines globalen Minimums der Zielfunktion. Bevor die idealen Ameisen mit dem Experiment anfangen können, wird $\mathcal{M}$ mit der Initialisierungsmethode $\mathcal{I}$ initialisiert. Hier wird jeder Matrixeintrag $(m_{ij})_{i,j=1,2,3}$ der Pheromonmatrix $\mathcal{M}$ auf Δ gesetzt, damit in den Gleichungen 1 und 2 nicht durch 0 geteilt wird. Das Experiment wird beendet, wenn das Stoppkriterium $\mathcal{S}$ erreicht ist. Das Stoppkriterium $\mathcal{S}$ ist hier, dass das Experiment, nach dem Verstreichen von 200 Zeiteinheiten beendet wird.

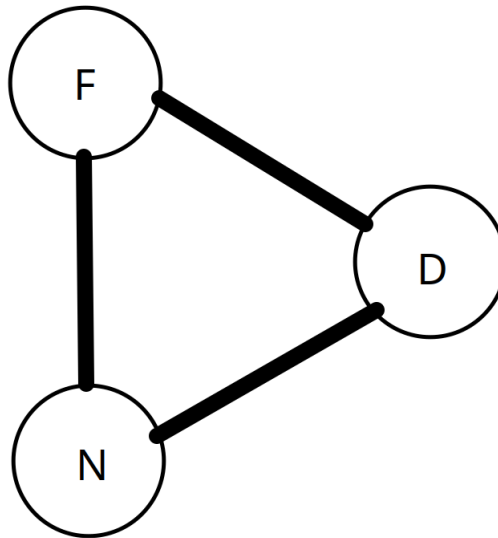


Abbildung 1: Experimentaufstellung
in Anlehnung an *Engelbrecht* [9]

Weiter, in Anlehnung an *Dorigo und Stützle* [8] und wie in meiner Bachelorarbeit [16] beschrieben, ergibt sich die Wahrscheinlichkeit, dass sich eine Ameise für das Begehen der Weges zwischen N und F entscheidet, aus der Gleichung 1 und die Wahrscheinlichkeit, dass eine Ameise sich für das Begehen des Weges zwischen N und D entschei-

det, aus Gleichung 2. Gleichung 3 beschreibt die Verdunstung des bereits vorhandenen Pheromons auf einem Weg nach dem eine Zeiteinheit verstrichen ist. Dabei ist ν die Verdunstungskonstante, welche bestimmt, wie schnell Pheromon nach jedem Zeitschritt verdunstet, und α ein Parameter, der bestimmt, wie viel Einfluss das auf einem Weg liegende Pheromon hat. Für Werte für α , nach [8], gilt $\alpha \in [0, 2]$. Wie viel Pheromon auf einem Weg liegt, also die Pheromoninformation, wird nach jedem Zeitschritt aktualisiert. Auf den Kanten, für die sich die Ameise entschieden hat, wird Pheromon abgelegt gemäß Gleichung 4.

$$prob_{NF} = \frac{\rho_{NF}^\alpha}{\rho_{NF}^\alpha + \rho_{ND}^\alpha} \quad (1)$$

$$prob_{ND} = \frac{\rho_{ND}^\alpha}{\rho_{ND}^\alpha + \rho_{NF}^\alpha} \quad (2)$$

$$\rho_{XY} = (1 - \nu) \cdot \rho_{XY}, \quad \nu \in [0, 1), \quad XY \in \{NF, ND, DF\} \quad (3)$$

$$\rho_{XY} = \rho_{XY} + \Delta_{XY} \quad (4)$$

Wie zu sehen, konstruiert das oben beschriebene Vorgehen eine Belegung der Knoten $\{N, F, D\}$ so, dass die Kürzeste Verbindung $\{FN\}$ als Ergebnis entsteht, $\{D\}$ also nicht begangen wird. Wie in meiner Bachelorarbeit [16] beschrieben, können wir beobachten, dass die Orte N, F und D nur begangen und nicht begangen als Werte annehmen können. Damit ist das Problem die kürzeste Verbindung zu finden, ein diskretes Problem, da die Werte, welche die Orte, verallgemeinert Variablen, annehmen können, abzählbar sind. Für das bis jetzt beschriebene Vorgehen wichtige Teilmenge von diskreten Problemen, sind Permutationsprobleme.

2.1.3. Permutationsprobleme

In Anlehnung an meine Bachelorarbeit [16], kann ein Permutationsproblem definiert werden durch das Tupel $(\mathcal{G}, \pi, \Sigma, \mathcal{B}, f)$. Für den Grundraum $\mathcal{G}$ gilt hier $\mathcal{G} = \{a_1, \dots, a_n\}, n \in \mathbb{N}_{>0}$, wobei die a_i beliebige Objekte sein können, die man ordnen kann. Die Permutation $\pi : \Sigma \rightarrow \Sigma$ ordnet jedem Element $a_i \in \Sigma, i \in \{1, \dots, n\}$ das korrespondierende Element $a_{\pi(i)}$ zu. Für den Suchraum Σ gilt hier $\Sigma = \{a_{\pi(1)}, \dots, a_{\pi(n)}\}$, wobei $\{a_{\pi(1)}, \dots, a_{\pi(n)}\}$ jede Permutation der Grundmenge $\{a_1, \dots, a_n\}$ ist. Die Menge an Beschränkungen, welche an die Ordnungen der $a_i, i \in \{1, \dots, n\}$ oder Werte von f gestellt werden, werden mit $\mathcal{B} = \{\mathcal{B}_1, \dots, \mathcal{B}_m\}, m \in \mathbb{N}_{>0}$ bezeichnet. Das Permutationsproblem ist unbeschränkt, wenn $\mathcal{B} = \emptyset$ gilt, sonst ist es beschränkt. Die Zielfunktion $f : \{a_{\pi(1)}, \dots, a_{\pi(n)}\} \rightarrow \mathbb{R}_{\geq 0}$ gibt die Güte einer Ordnung $\{a_{\pi(1)}, \dots, a_{\pi(n)}\}$ der Objekte an. Eine Lösung $s \in \Sigma$ ist eine Ordnung aller Elemente aus Σ , welche konform ist mit den Beschränkungen $\mathcal{B}$. Eine Lösung s_{opt} ist ein globales Optimum, wenn $f(s_{opt}) \leq f(s) \forall s \in \Sigma$ gilt. Ein wichtiger Spezialfall eines Permutationsproblems ist das Traveling Salesperson Problem (TSP).

2.1.4. Traveling Salesperson Problem (TSP)

Das TSP, nach *Stützle et al.* [23], ist ein Permutationsproblem $(\mathcal{G}, \pi, \Sigma, d, \mathcal{B}, f)$ mit $\mathcal{G} = \{a_i, \dots, a_n\}, n \in \mathbb{N}_{>0}$, wobei beim TSP die a_i die Städte sind und $d : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$ die Distanz zwischen zwei Städten ist. Da das TSP keine Beschränkungen hat, gilt $\mathcal{B} = \emptyset$. Die Permutation π gibt die Reihenfolge der Städte $\{a_{\pi(1)}, \dots, a_{\pi(n)}\}$ und damit eine Rundreise an. Eine Rundreise ist eine Reise, bei der jede Stadt, außer der Startstadt, nur einmal besucht wird. Die Zielfunktion f gibt die Länge der Rundreise $\{a_{\pi(1)}, \dots, a_{\pi(n)}\}$ an. Gesucht ist mindestens ein $s_{opt} \in \Sigma, s_{opt} = \{a_{\pi(1)}, \dots, a_{\pi(n)}\}$, sodass gilt $f(s_{opt}) \leq f(s) \forall s \in \Sigma$. Es ist also mindestens eine kürzeste Rundreise gesucht. Das TSP war eines der Probleme, dass sich angeboten hat, um mit der ACO-Metaheuristik gelöst zu werden.

2.2. ACO-Metaheuristik nach *Dorigo et al.* [7]

Die ACO-Metaheuristik, nach *Dorigo et al.* [7], kann als Erweiterung der Konstruktion aus Abschnitt 2.1.2 gesehen werden, da wir sie auch als Tupel $(\Sigma, \mathcal{B}, \mathcal{E}, \Delta, \mathcal{M}, \eta, f, \mathcal{I}, \mathcal{S})$ angeben können. Dabei ist zwar $\Sigma = G(V, E)$ aber V und E sind beliebig, genauso wie die Beschränkungen $\mathcal{B}$. Weiter gilt $|\mathcal{E}| = m, m \in \mathbb{N}$ und nicht nur $|\mathcal{E}| = 1$. Die Menge des Pheromons Δ , welche die iterationsbeste Ameise auf die von ihr gewählten Kanten hinterlegt, wird Problemabhängig gewählt. Die Pheromonmatrix $\mathcal{M}$ kann größer sein als im Abschnitt 2.1.2 aber sie ist genauso aufgebaut und auch ihre Funktionalität ist die selbe. Die problemabhängige Heuristik ist η . Die Zielfunktion f kann auch beliebig formuliert werden, es bleibt jedoch gleich, dass mindestens eines ihrer globalen Minima gesucht wird. Das Initialisierungskriterium bleibt gleich, wie im Abschnitt 2.1.2. Das Stoppkriterium $\mathcal{S}$ kann frei gewählt werden.

Für die ACO-Metaheuristik angewandt auf das TSP gilt kann $\mathcal{B} = \max_{v_i, v_j \in V} d(v_i, v_j) \leq D$ sein, wobei D die maximale erlaubte Distanz zwischen zwei Städten ist, wenn die maximale Tankfüllung eines Transporters oder andere Beschränkungen berücksichtigt werden müssen, sonst gilt $\mathcal{B} = \emptyset$. Die Anzahl der Ameisen ist $|\mathcal{E}| = m$. Die Menge des Pheromons Δ , dass von der iterationsbesten Ameise abgelegt wird, ist in der Gleichung 5 angegeben. Für das TSP wird η in der Gleichung 6 angegeben, wobei $d(v_i, v_j), v_i, v_j \in V$ die Distanz zwischen v_i und v_j ist.

$$\Delta_{ij} = \sum_{k=1}^m \frac{1}{f(s)}, (ij) \in s \quad (5)$$

Dabei ist f die im Abschnitt 2.1.4 definierte Zielfunktion, welche die Länge der Rundreise s wiedergibt. Wie zu sehen, ist die Menge des auf einer Kante (i, j) hinterlegten Pheromons größer, je kürzer die Rundreise s ist, in der die Kante (i, j) vorkommt.

$$\eta_{ij} := \frac{1}{d(v_i, v_j)} \text{ für zwei Knoten } v_i, v_j \in V \quad (6)$$

Wie in *Dorigo et al.* [7] beschrieben werden bei der Anwendung der ACO-Metaheuristik auf das TSP die Gleichungen 1 und 2 zuerst zur Gleichung 7 erweitert, wobei V' die Men-

ge der Städte ist, die noch nicht besucht wurden, danach zur Gleichung 8. Dabei hat α die selbe Funktion, wie in den Gleichungen 1 und 2 und β ist ein Parameter, der bestimmt, wie viel Einfluss die Heuristik η hat. Die Gleichung 8 ist schließlich die Gleichung, welche beim Algorithmus 1 im Schritt „CreateSolution(SCE)“ verwendet wird. In diesem Schritt wird eine Ameise zunächst auf einen zufällig ausgewählten Knoten gesetzt und von diesem Knoten aus wird dann mithilfe der Gleichung 8 der nächste Knoten gewählt. Dies wird solange wiederholt, bis $V' = \emptyset$ gilt.

$$prob_{ij} := \frac{\rho_{ij}}{\sum_{v \in V'} \rho_{iv}} \quad (7)$$

$$extend_prob_{ij} = \frac{\rho_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_{v \in V'} \rho_{iv}^\alpha \cdot \eta_{iv}^\beta} \quad (8)$$

Als letztes bemerken wir, dass „UpdatePheromoneInformation“, gegeben eine $n \times n$ -Matrix, da jeder Matrixeintrag aktualisiert werden muss, $\mathcal{O}(n^2)$ Schritte braucht, was bei großen Problemen viel Zeit verbraucht. Diese Herausforderung wurde durch *Guntsch und Middendorf* [10], mit der Weiterentwicklung der ACO-Metaheuristik zu der P-ACO-Metaheuristik, bewältigt.

Algorithm 1 ACO-Metaheuristik angewandt auf das TSP

```

1: repeat
2:   for SCE  $\in \mathcal{E}$  do
3:     CreateSolution(SCE)
4:   UpdatePheromoneInformation( $\mathcal{M}$ )
5: until  $\mathcal{S}$  fulfilled

```

2.3. Weiterentwicklung der ACO Metaheuristik zu der P-ACO-Metaheuristik

Die folgende Beschreibung ist nach *Guntsch und Middendorf* [10] und in Anlehnung an meine Bachelorarbeit [16].

P-ACO steht für Population based ACO. Die Unterschiede zu ACO sind, dass in P-ACO die Pheromoninformation nicht mehr in einer stetigen $n \times n$ -Pheromonmatrix $\mathcal{M}$, sondern in einer diskreten $n \times n$ -Pheromonmatrix $\mathcal{M}_d$ gespeichert wird. Die Matrix $\mathcal{M}_d$ ist diskret, da ihre Elemente nur endlich viele Werte annehmen können. In P-ACO wird auch zudem eine Liste $\mathcal{P}$ der Länge $k, k \in \mathbb{N}$, von Lösungen mitgeführt, wobei in [10] gilt $k \in \{1, 5, 25\}$. Die in $\mathcal{P}$ gespeicherten Lösungen werden als Populationen interpretiert, daher „P-ACO“ als Name für diese Erweiterung von „ACO“. Demnach können wir, angelehnt an ACO, die P-ACO-Metaheuristik als Tupel $(\Sigma, \mathcal{B}, \mathcal{E}, \Delta, \mathcal{M}_d, \mathcal{P}, \eta, f, \mathcal{I}, \mathcal{S})$

angeben. Nach [10], gilt $|\mathcal{E}| = 10$. Dabei wird in *Guntsch und Middendorf* [10] die Liste der Populationen nach dem Prinzip „first in last out“ befüllt. Bei diesem Prinzip wird, nachdem die Liste $\mathcal{P}$ zunächst mit k Lösungen befüllt wurde, wenn eine neue Lösung generiert wird, die älteste Lösung entfernt, die verbleibenden Lösungen um eine Stelle nach hinten verschoben und die neue Lösung an den ersten Platz in der Liste $\mathcal{P}$ gespeichert. Ein anderes Prinzip $\mathcal{P}$ zu verwalten ist das „best in worst out“ Prinzip. Dieses Prinzip wird im Abschnitt 2.4 erläutert. Das Befüllen von $\mathcal{P}$ mit k Lösungen und dem folgenden Aufbau von $\mathcal{M}_d$ ist hier die Initialisierungsmethode $\mathcal{I}$. Ein weiterer Unterschied ist, dass die Pheromonmatrix $\mathcal{M}_d$ nicht mehr beliebig viel Pheromoninformation enthält, wie $\mathcal{M}$, sondern jedes Element nur $k + 1$ verschiedene Werte annehmen kann, wodurch $\mathcal{M}_d$ diskret ist. Ferner muss bei P-ACO nicht mehr jedes Element der Pheromonmatrix $\mathcal{M}_d$ bei jeder Iteration betrachtet werden. Nur die Elemente von $\mathcal{M}_d$, welche in der neuen Lösung, welche in $\mathcal{P}$ aufgenommen wird, vorkommen und die in der alten Lösung vorkommen, welche aus $\mathcal{P}$ entfernt wird, und damit verändert werden, müssen betrachtet werden. Beides braucht $n, n \in \mathbb{N}$ Schritte, also insgesamt $2n$ Schritte. Damit braucht das Pheromonupdate statt $\mathcal{O}(n^2)$ Zeit, nur $\mathcal{O}(n)$. In dieser Tatsache liegt die Errungenschaft von *Guntsch und Middendorf* [10].

Genauer, nach [10] und [16], seien $\Delta > 0$, Δ konstant, $\rho_0 = 1/(n-1)$. Dann werden die Elemente der Pheromonmatrix $\mathcal{M}_d$ gemäß Gleichung 9 initialisiert. Nach [10] gilt weiter für die Gleichung 8, $\alpha = 1$ und $\beta = 5$. Die Elemente von $\mathcal{M}_d$ werden dann, wie in Gleichung 10 dargestellt, aktualisiert. Dabei darf, wie zuvor, nur die iterationsbeste Ameise Pheromon legen. Die Tatsache, dass jedes Element von $\mathcal{M}_d$ nur $k + 1$ verschiedene Werte annehmen kann, ergibt sich daraus, dass, auf der einen Seite, jedes Element mindestens aus dem Initialisierungswert ρ_0 besteht und, auf der anderen Seite, nur Vielfache von Δ hinzugefügt werden können und das nur maximal k -Mal, da die Menge an Pheromon, welches die iterationsbeste Ameise ablegen kann, konstant Δ ist und die Population der Ameisen $\mathcal{P}$, nach Initialisierung, konstant die Größe k hat. Somit kann eine Kante (i, j) nur maximal in k Lösungen vorkommen und damit ergibt sich, dass auf einer Kante nur $k + 1$ verschiedene Mengen an Pheromon liegen können. Dies wird nochmal in der Gleichung 11 verdeutlicht.

$$(m_{ij})_{i,j=1,\dots,n} = (\rho_{ij})_{i,j=1,\dots,n} = \rho_0; \quad (9)$$

$$\rho_{ij} = \rho_{ij} + \Delta \quad (10)$$

$$\rho_0 \leq \rho_{ij} \leq \rho_0 + k \cdot \Delta \quad (11)$$

Schließlich können wir noch festhalten, dass wir die Population $\mathcal{P}$ als Lösungsarchiv interpretieren können, mit dessen Hilfe neue Lösungen konstruiert werden können.

2.4. Ameisenalgorithmen Mit Einem Stetigen Lösungsarchiv

Ein anderes Prinzip $\mathcal{P}$ zu verwalten ist das „best in worst out“. Bei diesem Prinzip, wie in meiner Bachelorarbeit [16] beschrieben, wird Lösung nach ihrer Güte in die Liste $\mathcal{P}$ einsortiert. Ist die Güte der neu generierte Lösung schlechter als die Güte aller anderen Lösungen in $\mathcal{P}$, wird die neue Lösung verworfen. Ist die Güte der neuen Lösung besser als die Güte einer Lösung in $\mathcal{P}$, wird die neue Lösung an die Stelle der schlechteren Lösung gespeichert und alle Lösungen, die schlechter sind, als die neue Lösung werden um eine Stelle nach hinten verschoben

Wie im Abschnitt ?? beschrieben, kann man, statt die Pheromoninformation von Iterationsschritt zu Iterationsschritt mitzuführen, die Information auch in einer diskreten Matrix speichern, also einem diskreten Lösungsarchiv. Dies funktioniert jedoch nur für diskrete Probleme. Um stetige Probleme zu lösen, kann man auf ein Archiv mit stetigen Einträgen zurückgreifen, was dann u.a. eine Liste sein kann.

Ameisenalgorithmen mit einem stetigen Lösungsarchiv speichern ihre Lösungen in stetigen Einträgen. Wie genau, hängt von dem jeweiligen Algorithmus ab. Die einfachste Art und Weise dies zu tun, ist jedoch, die Lösungen wie sie generiert werden, nach einer bestimmten Vorschrift zu speichern, also ein einfaches Lösungsarchiv zu verwenden.

Wiederum die einfachsten Vorschriften zum Speichern von Lösungen sind „first in last out“ und „best in worst out“. Bei der ersten Vorschrift wird die neueste Lösung ins Archiv aufgenommen und die Älteste entfernt und bei der Zweiten wird die Lösung nach ihrer Güte einsortiert. Ist bei der zweiten Vorschrift die Lösung schlechter als alle anderen im Archiv, wird die Lösung verworfen, ist sie besser als eine gespeicherte Lösung, wird die neue gute Lösung an die Stelle der alten schlechteren Lösung gesetzt, alle Lösungen ab der schlechteren Lösung, die schlechtere Lösung eingeschlossen, werden um einen Platz nach hinten verschoben und die dann neue schlechteste Lösung wird aus dem Archiv gelöscht.

2.5. Ameisenalgorithmen Mit Einem Gemischten Lösungsarchiv

Da in dieser Arbeit die Art der Algorithmen, die getestet werden und die Art der Funktionen, die zum Testen benutzt werden, erweitert wird, müssen wir auch den Begriff des Lösungsarchivs erweitern. Gegeben eine Funktion mit gemischten Variablen, wie in den Abschnitten || und 9.4.1 und ein einfaches Lösungsarchiv mit $n \in \mathbb{N}$ Einträgen, wie es im Abschnitt 2.4 definiert wurde, dann ist die einfachste Erweiterung für ein einfaches Lösungsarchiv, mehrere einfache Lösungsarchive bzw. Listen zu einem einfachen zusammengesetzten Lösungsarchiv zusammenzufassen. Dabei bilden die Einträge der Unterarchive an der Stelle $i \in \{1, \dots, n\}$ eine Gesamtlösung. Zum Beispiel kann ein zusammengesetztes einfaches Lösungsarchiv $\{\{'diskret': [1], 'stetig': [1.0, 1.5]\}, \{'diskret': [2], 'stetig': [2.0, 3.0]\}\}$ sein. Dann ist die 2.te Lösung $\{'diskret': [2], 'stetig': [2.0, 3.0]\}$.

2.6. Ameisenalgorithmen mit einem Hierarchischen Gemischten Lösungsarchiv

Wie im Abschnitt 6 beschrieben, wird bei einem hierarchischen Algorithmus die Anzahl der Archive von insgesamt 1 auf 3 pro SCE erhöht. Ein Archiv für die letzte gefundene Lösung des SCE, Eines für die bis dato beste gefundene persönliche Lösung des SCE und Eines für die bis dato beste gefundene Lösung des Eltern-SCE. Zusätzlich gibt es noch 1 weiteres Archiv, welches die bis dato insgesamt beste Lösung hält. Die Größe der Lösungsarchive schrumpft aber, von $n = 100$ bei $ACO_{\mathbb{R}}$ -VS, auf $n = 1$. Dies ist notwendig, damit Lösungen entlang dem Baum im HSPBBO-Schema wandern können, denn man kann Archive nur dann einfach vergleichen, wenn sie nur eine Lösung haben. Das Vergleichskriterium ist dann der Funktionswert der Fitnessfunktion.

Eine offene Forschungsfrage ist, wie mit Archiven in einer hierarchischen Struktur umgegangen werden kann, wenn sie mehr als 1 Lösung halten. Die einfachste Anordnung ist nach der Besten Lösung im Archiv zu ordnen. Die zweit einfachste Anordnung ist die Archive nach der Durchschnittsqualität der Lösungen zu sortieren. Eine komplexere Möglichkeit die Archive zu ordnen ist einen zusammengesetzten Term zu verwenden, wie in der Gleichung 12 zu sehen. Dieser Term funktioniert sowohl für Archive, die Lösungen nach der Regel „first in last out“ speichern als auch für Archive, die Lösungen nach der Regel „best in worst out“ speichern. Im letzten Fall gilt $s_1 = s_{opt}$.

$$Güte = c_1 \cdot f(s_1) + c_2 \cdot \frac{\sum_{i=2}^n f(s_i)}{n - 1} \quad (12)$$

Dabei sind $c_1, c_2 \in \mathbb{R}$ Konstanten, s_{opt} ist die bis dato gefundene beste Lösung und n die Anzahl der Lösungen in den Archiven, wobei alle Archive die selbe Anzahl an Lösungen halten. In diesem Fall kann auf eine persönliche und globale beste Lösung verzichtet werden. Als Ausgangspunkt für weitere Forschung eignet sich mit der Beziehung $c_1 + c_2 = 1$, $c_1, c_2 \in [0, 1]$ anzufangen. Später kann man allgemeiner verlangen, dass nur $c_1, c_2 \in \mathbb{R}$ gilt. Als letzte Forschungsfrage können verschiedene Archive verschieden viele Lösungen halten.

2.7. Ameisenalgorithmen in stetigen Variablen

Ameisenalgorithmen in stetigen Variablen suchen Belegungen für jede ihrer Variablen in $\mathbb{R}$ oder in einem Intervall von $\mathbb{R}$, der Suchraum ist also z.B. $\mathbb{R}^n, n \in \mathbb{N}$.

Da für ein stetiges Problem keine Matrix mit diskreten Einträgen verwendet werden kann, ist es möglich, wie in *Socha und Dorigo* [22] und dem vorherigen Abschnitt beschrieben, ein Lösungsarchiv zu verwenden, in dem die Lösungen gespeichert werden.

Aus den Lösungen im Archiv wird dann eine neue Lösung konstruiert. Dies kann auf unterschiedlichste Weise passieren, wie in den Abschnitten ??, 4.2 und ?? beschrieben.

Das Beschreiben anderer Methoden der Generierung und Verarbeitung von Lösungen, abgesehen vom Speichern in einem Lösungsarchiv und der Konstruktion von neuen Lösungen aus diesem, geht über den Umfang dieser Arbeit hinaus und wird nicht betrachtet.

3. Optimierungsprobleme

Da wir, im Vergleich zu meiner Bachelorarbeit [16], in dieser Arbeit die Menge der betrachteten Probleme auf Probleme mit gemischten Variablen erweitern, bietet es sich an, eine abstraktere Definition von Optimierungsproblemen zu etablieren.

Wie an [15] angelehnt, definieren wir ein Optimierungsproblem durch 4 Teile. Diese sind der Suchraum $\Sigma = \{\Sigma_1, \dots, \Sigma_n\}, n \in \mathbb{N}$, die Menge der abstrakten Variablen $X = \{X_1, \dots, X_n\}, n \in \mathbb{N}$, die Beschränkungen an die Variablen $\mathcal{B} = \{\mathcal{B}_1, \dots, \mathcal{B}_n\}, n \in \mathbb{N}$ und eine Fitness- bzw. Zielfunktion $f : \Sigma \rightarrow \mathbb{R}^+$, die jeder Lösung $s \in \Sigma$ einen Wert $f(s) > 0$ zuordnet, wobei eine Lösung $s \in \Sigma$ das Belegen aller Variablen X_i mit den Werten aus Σ_i ist, sodass die Werte den Beschränkungen $\mathcal{B}_i$ genügen, $i \in \{1, \dots, n\}$. Damit kann ein Optimierungsproblem definiert werden als das Tupel $(\Sigma, X, \mathcal{B}, f)$. Wenn $\mathcal{B} = \emptyset$, dann ist das Problem unbeschränkt, sonst ist es beschränkt.

Eine Population $\mathcal{P}$ besteht aus mindestens einer Lösung $s \in \Sigma$. Alle Lösungen $s \in \Sigma$, die auch in einer Population $\mathcal{P}$ sind, werden durch $\Sigma|\mathcal{P}$ beschrieben. Darauf aufbauend, gegeben eine Population $\mathcal{P}$, beschreibt $\Sigma|\mathcal{P}_i, i \in \{1, \dots, n\}$, alle Werte in Σ an der Stelle i , die auch in $\mathcal{P}$ an der Stelle i auftreten.

Eine Lösung s_{opt} heißt Optimum, wenn gilt $f(s_{opt}) \leq f(s) \forall s \in \Sigma$. Ein Optimierungsproblem kann mehr als ein s_{opt} haben. Das Ziel eines Optimierungsproblems ist mindestens ein s_{opt} zu finden.

Mit dieser nun rigoroseren Definition können wir beispielhaft den Suchraum und die Variablen für das Coil Spring Design Problem angeben. Der Suchraum ist $\Sigma = \{\mathbb{R}, \mathbb{O}, \mathbb{N}_{>0}\}$ und die Variablen sind $X = \{D, d, N\}$. $\mathbb{O}$ ist die Menge aller rationalen ordinalen Zahlen. Es ist die Menge der rationalen Zahlen $\mathbb{Q}$ ohne eine Ordnungsrelation. D ist eine stetige Variable, d ist ordinal und N ist natürlich, wobei die Beschränkungen $D \in \{3 \cdot d_{min}, 3.0\}$, $d \in \{d_{min}, \dots, 0.5000\}$ und $N \in \{1, \dots, 70\}$ gelten. Die Beschränkung d_{min} wird im Abschnitt 9.4.1 zum Coil Spring Design Problem (CSD Problem) angegeben.

4. Grundlagen

Das dieser Arbeit beiliegende Programm ist ein Intervallauswertungsalgorithmus. Wie in der Einleitung motiviert, fragen wir uns, ob komplexe Lösungsfindungsmethoden wirklich nötig sind, um gute Lösungen zu finden, unabhängig davon, ob wir bereits vorhan-

dene Lösungen zu Neuen zusammensetzen oder komplett neue Lösungen auf eine andere Art und Weise generieren.

Deswegen, anstatt wie *Socha und Dorigo* [22], wo eine Funktion über einem Intervall aufgebaut und dann ausgewertet wird, nehmen wir einfach das Intervall und werten in gewisser Weise nur das Intervall selbst aus. Dabei nehmen wir einen zufällig bestimmten Punkt aus dem Intervall und nennen dies Auswerten. Daher auch Intervallauswertungsalgorithmus.

Insbesondere sei hier festgehalten, dass dies als Variante der Lösungserzeugung interpretiert wird, also dass wir ein Intervall nehmen und dann daraus einen zufälligen Punkt und nicht die Methode mit der wir das Intervall berechnen, davon gibt es nämlich zwei. Die andere Variante ist das Aufbauen einer Funktion über einem Intervall und anschließend das Auswerten dieser Funktion, was in [22] durch $\text{ACO}_{\mathbb{R}}$ getan wird. Daher auch der Titel dieser Bachelorarbeit – es werden zwei Varianten der Lösungserzeugung verglichen, nicht Algorithmen.

Die Variante zur Lösungserzeugung, durch das Auswerten einer Funktion, wird im Abschnitt ?? beschrieben. Die Variante zur Lösungserzeugung durch das Auswerten eines Intervalls folgt, realisiert durch die beiden Algorithmen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple, in den Abschnitten 4.2 und 4.3.

4.1. $\text{ACO}_{\mathbb{R}}$

Wie in *Socha und Dorigo* [22] dargelegt, wird in $\text{ACO}_{\mathbb{R}}$ ein Lösungsarchiv bei der Lösungssuche mitgeführt. Das Archiv hat die Größe k , es hält also zu jedem Zeitpunkt k Lösungen.

Des Weiteren führt in $\text{ACO}_{\mathbb{R}}$ jede Ameise n Konstruktionsschritte durch, wobei n die Anzahl der Dimensionen des Problems ist. Dabei wird im i -ten Konstruktionsschritt die folgende Funktion ausgewertet.

$$G^i = \sum_{l=1}^k \omega_l g_l^i(x) = \sum_{l=1}^k \omega_l \frac{1}{\sigma_l^i \sqrt{2\pi}} e^{-\frac{(x-\mu_l^i)^2}{2\sigma_l^{i2}}}, \quad i \in \{1, \dots, n\} \quad (13)$$

Für Vektoren ω , μ und σ gilt, dass sie so viele Komponenten haben, wie es Lösungen im Lösungsarchiv gibt, also $|\omega| = |\mu| = |\sigma| = k$.

Ferner werden die Parametervektoren ω , μ und σ folgendermaßen berechnet.

$$\mu = \{\mu_1^i, \dots, \mu_k^i\} = \{s_1^i, \dots, s_k^i\}, \quad i \in \{1, \dots, n\}, \quad (14)$$

wobei n die Anzahl der Dimensionen ist, i der Konstruktionsschritt und s_j^i die i -te Variable der j -ten Lösung ist. Die Gewichte für jede Lösung ergeben sich nach Gleichung (15).

$$\omega_l = \frac{1}{qk\sqrt{2\pi}} e^{-\frac{(l-1)^2}{2q^2k^2}}, \quad l \in \{1, \dots, k\}, \quad (15)$$

Demnach wird die l -te Gaußfunktion nach der Wahrscheinlichkeitsverteilung aus Gleichung (16) gewählt.

$$p_l = \frac{\omega_l}{\sum_{r=1}^k \omega_r}, \quad l \in \{1, \dots, k\} \quad (16)$$

Und die Standardabweichung im i -ten Konstruktionsschritt wird wie folgt berechnet.

$$\sigma_l^i = \xi \sum_{e=1}^k \frac{|s_e^i - s_l^i|}{k-1}, \quad l \in \{1, \dots, k\}, \quad i \in \{1, \dots, n\}. \quad (17)$$

wobei q und ξ Parameter sind. Es mag zwar die Wurzel fehlen aber in *Socha und Dorigo* [22] wurde die Standardabweichung so berechnet bzw. der Term in der Gleichung (17) als Standardabweichung bezeichnet.

Abschließend wird in Schritt i die Gaußfunktion g_l^i ausgewertet. Dies kann u.a. durch einen Zufallszahlengenerator geschehen, der Zufallszahlen gemäß einer parametrisierten Normalverteilung generieren kann.

Abschließend gehen wir noch auf die Parameter q und ξ ein. Der Parameter q beeinflusst, wie die Lösungen im Archiv in die Suche eingebunden werden. Bei kleinem q werden besseren Lösungen stärker bevorzugt. Und der Parameter ξ beeinflusst, wie schnell der Algorithmus konvergiert. Je größer ξ , desto langsamer konvergiert der Algorithmus.

4.2. ACO_ℝ-simple

Der erste dieser Arbeit beiliegende Algorithmus ACO_ℝ-simple, nutzt das Lösungsarchiv auch wie ACO_ℝ, jedoch setzt es die vorhandenen Lösungen auf eine sehr viel einfachere Weise zu einer neuen Lösung zusammen – deswegen simple, also einfach.

Dabei wird, wie in ACO_ℝ dieselbe Gewichtung der Lösungen vorgenommen. Zuerst werden die Gewichte anhand Gleichung (18) berechnet,

$$\omega_i = \frac{1}{qk\sqrt{2\pi}} e^{-\frac{(i-1)^2}{2q^2k^2}}, \quad i \in \{1, \dots, k\}, \quad (18)$$

wobei auch hier k die Anzahl der Lösungen im Lösungsarchiv ist, also die Größe des Archivs. Danach wird die Wahrscheinlichkeitsverteilung, nach welcher eine Lösung aus dem Archiv gewählt wird, gemäß Gleichung (19) berechnet.

$$p_i = \frac{w_i}{\sum_{j=1}^n w_j}, \quad i \in \{1, \dots, k\}. \quad (19)$$

Es wird also zufällig eine Lösung s_i mit der Wahrscheinlichkeit p_i aus dem Lösungsarchiv gewählt. Danach wird die lineare Abweichung in der d -ten Dimension zu dieser Lösung mit der Gleichung (20) berechnet.

$$\delta_d = \xi \sum_{j=1}^k \frac{|s_{j-d} - s_{i-d}|}{k-1}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\}, \quad (20)$$

Dabei ist n die Anzahl der Dimensionen des Problems.

Anschließend wird für jede Dimension ein zufälliger Punkt aus dem Intervall

$$[s_d - \delta_d, s_d + \delta_d], \quad d \in \{1, \dots, n\}, \quad (21)$$

genommen. Damit haben wir, ausgehen von einem Lösungsarchiv, eine neue Lösung gewonnen.

Die Standardeinstellungen der Parameter sind, wie in Tabelle 1 zu sehen ist.

Tabelle 1: Parametereinstellungen $\text{ACO}_{\mathbb{R}}$ -simple

Parameter	Symbol	Wert
Anzahl der Ameisen	m	1
Konvergenzgeschwindigkeit	ξ	0.95
Lokalität der Suche	q	0.95
Archivgröße	k	100

100 da Algorithmus einfacher und er damit braucht mehr Information/Lösungsinformation

4.3. $\text{ACO}_{\mathbb{R}}$ -very-simple

Der zweite dieser Arbeit beiliegende Algorithmus $\text{ACO}_{\mathbb{R}}$ -very-simple, nutzt ebenfalls das Lösungsarchiv wie $\text{ACO}_{\mathbb{R}}$, jedoch setzt es die vorhandenen Lösungen noch einfacher, geradezu trivial, zu einer neuen Lösung zusammen – deswegen very-simple, also sehr einfach.

Bei diesem Algorithmus wird immer nur die erste, also die beste Lösung aus dem Archiv zur Intervallberechnung benutzt, also entfällt alles, bis auf die Intervallberechnung und das Bestimmen eines zufälligen Punktes aus dem errechneten Intervall. Das heißt, die

Gewichtung der Lösungen entfällt, genau wie das Berechnen der Wahrscheinlichkeitsverteilung für das Auswählen einer Lösung aus dem Lösungsarchiv.

Es wird also, wie oben erwähnt, immer nur die erste Lösung s_1 aus dem Lösungsarchiv gewählt. Danach wird die lineare Abweichung in der d -ten Dimension zu dieser Lösung mit der Gleichung (22) berechnet.

$$\delta_d = \xi \sum_{j=1}^k \frac{|s_{j_d} - s_{1_d}|}{k-1}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\}, \quad (22)$$

Dabei ist n die Anzahl der Dimensionen des Problems.

Anschließend wird für jede Dimension wieder ein zufälliger Punkt aus dem Intervall

$$[s_d - \delta_d, s_d + \delta_d], \quad d \in \{1, \dots, n\}, \quad (23)$$

genommen. Damit haben wir auch hier, ausgehen von einem Lösungsarchiv, eine neue Lösung gewonnen.

Die Standardeinstellungen der Parameter sind, wie in Tabelle 2 zu sehen ist.

Tabelle 2: Parametereinstellungen ACO_ℝ-very-simple

Parameter	Symbol	Wert
Anzahl der Ameisen	m	1
Konvergenzgeschwindigkeit	ξ	0.95
Archivgröße	k	100

Da ACO_ℝ-simple den Lösungsfindungsprozess zwar deutlich vereinfacht, jedoch einen integralen Bestandteil, das Generieren von neuen Lösungen ausgehend von weniger guten Lösungen beibehält, also anstatt nur stur nach der besten Lösung zu gehen, auch in anderen Gebieten nach vielversprechenden Ergebnissen sucht, ist dieser Algorithmus, gemäß unserer Intuition, der Favorit zwischen den beiden Algorithmen zur Intervallauswertung.

4.4. Erweiterung von ACO_ℝ-very-simple für Probleme mit nicht-stetigen Variablen

Da ACO_ℝ-VS ursprünglich für Funktionsoptimierung von stetigen Problemen entwickelt wurde, muss die ACO_ℝ-VS-Metaheuristik für diskrete/ordinale Probleme und für Probleme mit gemischten Variablen angepasst werden.

Um ACO_ℝ-VS anzupassen werden nicht-stetige Probleme als stetige Probleme behandelt. Das heißt, es wird mit nicht-stetigen Werten gerechnet als wären sie stetig. Es wird

also bis zur Gleichung 23 genauso verfahren, wie im Abschnitt 4.3, um einen neuen, stetigen Wert zu erhalten. Nachdem man einen neuen stetigen Wert gewonnen hat, betrachtet man die nicht-stetigen Werte zwischen denen der stetige Wert liegt und wählt dann als Ergebnis, den nicht-stetigen Wert, an dem der Stetige am nächsten liegt.

5. SPPBO-Schema

Wie in [15] beschrieben, besteht das SPPBO-Schema aus 6 Teilen. Diese sind der Suchraum Σ , die Menge der lösungskreierenden Entitäten $\mathcal{E}$, die Menge der Populationen bzw. Archive von Lösungen $\mathcal{P}$, die Heuristik η , die Initialisierungsmethode $\mathcal{I}$ und das Stoppkriterium $\mathcal{S}$. Also kann das SPPBO-Schema beschrieben werden durch das Tupel $(\Sigma, \mathcal{E}, \mathcal{P}, \eta, \mathcal{I}, \mathcal{S})$. Im Folgenden werden die lösungskreierenden Entitäten als SCE (solution creating entities) bezeichnet.

Der Suchraum Σ kann $\Sigma = \{a_1, \dots, a_n\}$ sein, für das TSP, wobei die $a_i, i \in \{1, \dots, N\}$ die Positionen der Städte sind, $\mathbb{R}^n, n \in \mathbb{N}$ für stetige Funktionen, wie der Paraboloidfunktion, wie sie in Tabelle 3 definiert ist oder eine Zusammensetzung von Intervallen über verschiedenen Skalenniveaus, wie es in Abschnitt 9.4.1 für das CSD ist, wo Σ sich aus einem stetigem, einem ordinalen und einem Intervall über den natürlichen Zahlen zusammensetzt. Im SPPBO-Schema kann $\mathcal{E}$ aus 2 Ameisen, wie in ACO [2] oder PACO [10], bestehen oder auch nur einer Ameise, wie in ACO_ℝ-VS [16], oder $N \in \mathbb{N}$ Partikeln, wie im Set-Based Discrete PSO [5]. Die Menge der Populationen $\mathcal{P}$ im SPPBO-Schema, wie es in [15] definiert ist, besteht aus zwei Populationen, die jede SCE mitführt, einer Persönlichen P_{pers}^{SCE} und einer Globalen P_{best}^{SCE} . Die persönliche Population p_{pers} besteht aus der besten Lösung, welche die jeweilige SCE bis einschließlich der letzten Iteration des Algorithmus gefunden hat. Die globale Population P_{best}^{SCE} besteht aus der besten Lösung, die alle SCE bis einschließlich der letzten Iteration gefunden haben. Wenn es jedoch nur eine Population, wie in ACO_ℝ-VS [16], gibt, dann kann diese $n \in \mathbb{N}$ Lösungen haben, wobei $n = 100$ in [16] gewählt wurde.

Beim Ausführen des SPPBO-Schema werden mit Hilfe der Initialisierungsmethode $\mathcal{I}$ zunächst Lösungen aus dem Suchraum Σ erzeugt, welche die Initiallösungen der Populationen bilden. Sollte es Einschränkungen/Constraints für Σ geben, wie es für das CSD Problem ist, können diese gleich beim Erzeugen der Initiallösungen berücksichtigt werden, sodass nur gültige Lösungen aus Σ generiert werden. Sollte sich für dieses Vorgehen entschieden werden, können zusätzliche Iterationen bei der Initialisierung benötigt werden, neben den Iterationen bei der Ausführung der Lösungssuche des Algorithmus. In diesem Fall bezeichnen wir die Initialisierungsmethode als $\mathcal{I}_{exact}$. Wenn das Filtern des Suchraumes Σ nach gültigen Lösungen bei gegebenen Einschränkungen während der Initialisierung nicht durchgeführt wird, können die Suchvariablen mit Werten aus Σ so belegt werden, dass keine gültigen Lösungen entstehen. Deswegen wird, sollte der Suchraum Σ nicht gefiltert werden, eine Straffunktion eingeführt, welche sich aus der Zielfunktion und Straftermen zusammensetzt. In diesem Fall bezeichnen wir die Initialisierungsmethode

als $\mathcal{I}_{penalty}$. Bei der Anwendung von $\text{ACO}_{\mathbb{R}}\text{-VS}$ auf das CSD Problem werden beide Vorgehen betrachtet und miteinander verglichen. Algorithmus 2 zeigt nochmal den Ablauf des SPPBO-Schema in Pseudocode.

Algorithm 2 SPPBO

```

1: Execute  $\mathcal{I}$ 
2: repeat
3:   for  $\text{SCE} \in \mathcal{E}$  do
4:     CreateSolution(SCE)
5:   for  $P \in \mathcal{P}$  do
6:     UpdatePopulation(P)
7: until  $\mathcal{S}$  fulfilled

```

6. HSPPBO-Schema

Das HSPPBO-Schema steht für Hierarchical Simple Probabilistic Populations-Based Optimization-Schema. Es ist die hierarchische Version des SPBBO-Schema [15]. Das HSPPBO-Schema, wie in [13] beschrieben, besteht aus 11 Teilen, den 6 Teilen aus dem SPPBO-Schema und einer Hierarchie- bzw. Baumstruktur $\mathcal{B}$, einer Funktion *range* $\mathcal{R}$, welche jeder $\text{SCE} \in \mathcal{E}$ eine Menge von Populationen $P \in \mathcal{P}$ zuweist, also $\text{range}(A) \subseteq \mathcal{P}$, eine Funktion *update* $\mathcal{U}$, welche bestimmt, wie Knoten getauscht werden, wenn ein Kindknoten eine bessere Lösung findet, als der Elternknoten, einem Threshold θ , der bestimmt, wann der Algorithmus auf Veränderungen im SCE-Baum reagiert und eine Funktion *handling* $\mathcal{H}$, welche bestimmt, wie der Algorithmus auf Veränderungen reagiert. Demnach kann das HSPPBO-Schema beschrieben werden durch das Tupel $(\Sigma, \mathcal{E}, \mathcal{B}, \mathcal{R}, \mathcal{U}, \theta, \mathcal{H}, \mathcal{P}, \eta, \mathcal{I}, \mathcal{S})$. In der Ausführung in [13] weist $\mathcal{R}$ jeder $\text{SCE} \in \mathcal{E}$ drei Populationen zu. Eine Population $P_{persprev}^{SCE}$ hält die von der SCE als letztes berechnete Lösung des Optimierungsproblems, eine Population $P_{persbest}^{SCE}$ hält die bis dato beste berechnete Lösung des SCE und eine Population $P_{parentbest}^{SEC}$ hält bis dato beste berechnete Lösung der Eltern-SCE. Es gilt also $P_{parentbest}^{SEC} = P_{persbest}^{parent(SEC)}$. Im Fall der Wurzel-SCE sind die letzten beiden Populationen identisch. Damit gibt es implizit eine globale beste Lösung, da die Population $P_{persbest}^{SCE^*}$ der Wurzel SCE^* ausgelesen werden kann.

Das HSPPBO-Schema zeichnet sich dadurch aus, dass die SCE in einer Baumstruktur angeordnet sind. Dadurch wird eine Nachbarschaftstopologie ermöglicht, welche die SCE gemäß der Güte ihrer Lösung ordnet. In [13] wurde sich für eine tertiäre Baumstruktur mit 13 Knoten entschieden, die wir hier übernehmen. Wie beschrieben ordnet $\mathcal{R}$ jeder der SCE drei Populationen zu, die alle nur eine Lösung haben. Von diesen Populationen ist für die Funktion $\mathcal{U}$ nur die Population $P_{persbest}^{SCE}$ entscheidend. Stellt der Algorithmus fest, dass $P_{persbest}^{SCE} > P_{persbest}^{parent(SCE)}$ gilt, werden die SCE im Baum getauscht. Die Prüfung,

ob dies gilt, wird „top-down“ durchgeführt. Wie in [13] beobachtet, lässt dieses Vorgehen schlechte Lösungen schnell im Baum sich nach unten bewegen, während gute Lösungen hinreichend lange gut bleiben müssen, um im Baum nach oben zu wandern. Wird festgestellt, dass die Anzahl der Vertauschung im Baum im Verhältnis zur Gesamtanzahl an SCE im Baum einen Threshold θ übersteigt, wird die Funktion $\mathcal{H}$ aufgerufen. Nach [13] gilt für den Threshold $\theta \in \{0.1, 0.25, 0.5\}$. Im Code ist $\theta = 0.25$ eingestellt. Die Einstellung wurde beibehalten. Wird der Threshold überschritten, greift $\mathcal{H}$. Nach [13] gibt es drei Einstellungen für $\mathcal{H}$, nämlich $\mathcal{H}_{full}$, $\mathcal{H}_{partial}$ und $\mathcal{H}_{none}$. Ist $\mathcal{H}_{full}$ eingestellt, werden beim überschreiten des Thresholds alle Populationen $P_{persbest}^{SCE}$ gelöscht. Das entspricht einem kompletten Vergessen der SCE und ist äquivalent zum Neustart des Algorithmus. Ist $\mathcal{H}_{partial}$ eingestellt, werden nur die Population $P_{persbest}^{SCE}$ der SCE ab der Tiefe 3 und abwärts gelöscht. Ist $\mathcal{H}_{none}$ eingestellt, gibt es keine Veränderungen an den Populationen. Im Code ist $\mathcal{H}_{partial}$ eingestellt. Die Einstellung wurde beibehalten.

Algorithmus 3 zeigt nochmal den Ablauf des HSPPBO-Schema in Pseudocode.

Algorithm 3 HSPPBO

```

1: Initialize  $\mathcal{B}$ 
2: Execute  $\mathcal{I}$ 
3: repeat
4:    $L \leftarrow L + 1$ 
5:   for  $|\mathcal{E}|$  do
6:     NumberOfSwaps = 0
7:     CreateSolution(SCE)
8:     UpdatePopulation( $P_{persprev}^{SCE}$ )
9:     UpdatePopulation( $P_{persbest}^{SCE}$ )
10:  for  $|\mathcal{E}|$  do
11:    if  $f(P_{persbest}^{parent(SCE)}) < f(P_{persbest}^{SCE})$  then
12:      swap tree position between SCE and parent(SCE)
13:      NumberOfSwaps  $\leftarrow$  NumberOfSwaps + 1
14:  if NumberOfSwaps  $> \lceil \theta \cdot |\mathcal{E}| \rceil$  and  $L_{pause} > 5$  then
15:     $L \leftarrow 0$ 
16:    Execute  $\mathcal{H}$ 
17: until  $\mathcal{S}$  fulfilled

```

7. Formulierung Von $\text{ACO}_{\mathbb{R}}$ -VS Im HSPPBO-Schema

8. Kombinieren des HSPPBO-Schemas mit $\text{ACO}_{\mathbb{R}}$ -very-simple

8.1. Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -very-simple

Die einfachste Kombination aus dem HSPPBO-Schema und $\text{ACO}_{\mathbb{R}}$ -very-simple ergibt Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -very-simple. Dabei führt jede SCE, nach der Initialisierung, den $\text{ACO}_{\mathbb{R}}$ -very-simple Algorithmus aus. Das Archiv, welches bei der Ausführung von $\text{ACO}_{\mathbb{R}}$ -very-simple notwendig ist, hat diesmal statt 100 Lösungen nur 4. Diese 4 Lösungen sind die 3 Populationen, die jede der SCE mitführt plus die Population, welche die beste bis dato gefundene Lösung hält, wie in Abschnitt 6 erläutert. Der Mechanismus, welcher das Tauschen von Populationen steuert, wenn eine Eltern-SCE eine schlechtere Lösung oder eine Kind-SCE eine bessere Lösung findet, wie ebenfalls im Abschnitt 6 dargelegt, bleibt der gleiche. Dies führt zur Bezeichnung Hierarchical Population-Based $\text{ACO}_{\mathbb{R}}$ -very-simple oder kurz HPB- $\text{ACO}_{\mathbb{R}}$ -VS. Was wir noch nicht aufgegriffen haben und wovon in [13], über das Tauschen der Knoten im Baum hinaus, kein weiterer Gebrauch gemacht wird, ist, dass die SCEs, abhängig von ihrer Position im Baum, nicht exakt denselben Algorithmus ausführen müssen. Weil aber die erste Iteration der Kombination des HSPPBO-Schemas mit dem $\text{ACO}_{\mathbb{R}}$ -very-simple Algorithmus dies ebenfalls nicht ausnutzt, bezeichnen wir diese Iteration Homogen-HPB- $\text{ACO}_{\mathbb{R}}$ -VS oder kurz Hom-HPB- $\text{ACO}_{\mathbb{R}}$ -VS.

Im HPB- $\text{ACO}_{\mathbb{R}}$ -VS

8.2. Het-HPB- $\text{ACO}_{\mathbb{R}}$ -very-simple

$\text{ACO}_{\mathbb{R}}$ -Very-Simple hat in seiner Definition im Abschnitt 4.3, Gleichung 24 einen Parameter, der sich anbietet, um, abhängig von der Position der SCE im tertiären Baum, mit der Höhe im Baum zu variieren. Dieser Parameter ist ξ . Wir greifen die Gleichung 24 auf.

$$\delta_d = \xi \sum_{j=1}^k \frac{|s_{j-d} - s_{1-d}|}{k-1}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\}, \quad (24)$$

Für die Tiefe 0, also die Wurzel, setzen wir $\xi = 0.7$, für die Tiefe 1 setzen wir $\xi = 1.0$, für die Tiefe 2 setzen wir $\xi = 1.3$ und ab der Tiefe 3 setzen wir $\xi = 1.5$.

Die Entscheidung die initialen Werte von ξ so zu wählen, ergibt sich aus der Überlegung, dass je höher im Baum sich die SCE befindet, desto höher auch die Güte der Lösung dieses SCE. Ferner gilt für viele Optimierungsprobleme, dass in der Nähe schlechter Lösungen sich weitere schlechte Lösungen befinden und in der Nähe guter Lösungen sich

weitere gute Lösungen befinden. Daher verkleinert ξ das Suchintervall für die Tiefe 0, lässt es unverändert für die Tiefe 1 und vergrößert es ab Tiefe 2.

Wir müssen jetzt jedoch die Generierung des neuen Punktes in Gleichung 23 anpassen, da für $\xi > 1.0$ das Intervall größer sein kann als Initialisierungsintervall, was dazu führen kann, dass eine Lösung generiert wird, die nicht im Definitionsraum des Problems liegt und damit ein Optimum angenommen werden kann, dass so nicht existiert. Deswegen ändern wir die Gleichung 23 zur Gleichung 25, wobei lg_d und rg_d die respektiven linken bzw. rechten Intervallgrenzen der d -ten Dimension des Testproblems sind.

$$[\max(s_d - \delta_d, lg_d), \min(s_d + \delta_d, rg_d)], \quad d \in \{1, \dots, n\}, \quad (25)$$

8.3. Het-HSPPB-ACO_ℝ-very-simple

Schließlich wird noch die Zufallskomponente in Hetero-HPB-ACO_ℝ-very-simple eingearbeitet. Auch hier bietet es sich an, abhängig von der Position der SCE im Baum, die Wahrscheinlichkeit mit der die Zufallskomponente verwendet wird, zu verändern. Die Zufallskomponente für die sich entschieden wurde, ist das Verwenden einer zufälligen Lösung aus dem Initialisierungsraum, anstatt der Lösungskonstruktion in Gleichung 24. Diese zufällige Lösung wird in der Tiefe 0 mit der Wahrscheinlichkeit 0% genommen, in der Tiefe 1 mit 10%, in der Tiefe 2 mit 15% und in der Tiefe 3 mit der Wahrscheinlichkeit 20%.

9. Testfunktionen

Aufbauend auf meiner Bachelorarbeit [16], erweitern wir die Art der Testfunktionen, anhand derer wir die Güte der hier vorlegten Erweiterungen von ACO_ℝ-VS beurteilen. Wo ACO_ℝ-VS nur auf stetigen Testfunktionen getestet wurde, werden die Erweiterungen von ACO_ℝ-VS auf stetigen Testfunktionen getestet, auf Testfunktionen mit diskreten/ordinalen Variablen und auf Testfunktionen mit gemischten Variablen.

Da in meiner Bachelorarbeit die Testfunktionen in ihren Dimensionen nicht variiert haben, Intervallgrenzen keine Rolle spielten und alle Funktionen über stetigen, zusammenhängenden Definitionsbereichen definiert waren, brauchte es keiner standardisierten, abstrakten Definition. In dieser Arbeit aber betrachten wir stetige und diskrete/ordinale Testfunktionen, sowie Testfunktionen mit gemischten Variablen. Ferner liegen 5 der Testprobleme mit gemischten Variablen in drei verschiedenen Dimensionen dar, nämlich $n \in \{2, 5, 10\}$. Deswegen war eine überarbeitete Definition der Testprobleme notwendig. Die einfachste Definition mit dem neuen Standard ist im Code-Snipped 1 zu sehen.

Code-Snipped 1: Paraboloid 10 Dim stetiger Definitionsbereich

```

1 {
2     "name": "Paraboloid",
3     "comment": "standard paraboloid problem",
4     "type": "simple continuous problems",
5     "dimension": 10,
6     "optimum_argument": [[0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
7                           0.0, 0.0, 0.0, 0.0]],
8     "optimum": 0,
9
10    "variable_boundaries": {
11        "natural_numbers": [],
12        "discrete": [],
13        "continuous": [[-3, 7]],
14        "ordinal": [],
15        "categorical": []
16    },
17
18    "discrete_values": [],
19
20    "magnitudes_of_categorical_variables": []
21 }
```

Wie im Code-Snipped 1 zu sehen, besteht die neue, abstrakte Definition u.a. aus „variable_boundaries“. Dies ist ein neuer Eintrag, der notwendig geworden ist, da für das Problem im Abschnitt 9.4.1 die Intervallgrenzen zugleich Beschränkungen der Suchintervalle sind. Ein weiterer Grund das JSON-Format zu nehmen, ist, dass alle Einträge als Wörterbücher (Dictionary) ausgelesen werden können, was die Definition von neuen abstrakten Einträgen leicht macht, da die Anordnung, im Vergleich zu einem Feld, bei einem Wörterbuch keine Rolle spielt.

Weitere Einträge, die notwendig wurden, sind „dimension“, „optimum“, „discrete_values“ und „magnitudes_of_categorical_variables“. Da ein weiteres Ziel war, dass, neben der Abstraktion, Automatisierung implementiert werden sollte, wurden die JSON-Dateien so konzipiert, dass statische Werte einer Testfunktion, ohne Nutzereingabe, zum Start des Algorithmus ausgelesen werden konnten. Dabei bezeichnet „dimension“ die Dimension des Problems. In Kombination mit den „variable_boundaries“ ergibt dies für den Code-Snipped 1 den Definitionsbereich $[-3, 7]^{10}$. Der Eintrag „optimum_argument“ definiert das zum Optimum gehörige Argument, also die notwendige Belegung der unabhängigen Variablen, um das Optimum der Testfunktion zu erhalten, welches im Eintrag „optimum“ ausgelesen werden kann. Die Intervallgrenzen für den Definitionsbereich einer Testfunktion sind unter „variable_boundaries“ hinterlegt und definieren sowohl die Intervalle, in

denen eine Lösung liegen muss, um eine gültige Lösung zu sein, als auch die Intervalle, aus denen Initiallösungen während der Initialisierungsmethode $\mathcal{I}$ gezogen werden, wie diese im Abschnitt 6 definiert wurde. Der Eintrag „discrete_values“ listet alle diskreten/ordinalen Werte auf, welche die Testfunktion annehmen kann

Der letzte Grund, warum das JSON-Format gewählt wurde, wie im Code-Snipped 1 zu sehen, kann das JSON-Format jede Art von Eintrag aufnehmen und somit jede Form von abstrakter Variable beherbergen und damit einen hohen Abstraktionsgrad liefern, der, für auf dieser Arbeit aufbauende Forschung, notwendig ist.

9.1. Permutationsprobleme Testfunktionen

9.1.1. Traveling Salesperson Problem (TSP)

9.1.2. Quadratic Assignment Problem (QAP)

9.2. Stetige Testfunktionen

Aufbauend auf meiner Bachelorarbeit [16] werden die Erweiterungen von $\text{ACO}_{\mathbb{R}}$ -very-simple ($\text{ACO}_{\mathbb{R}}$ -VS) auf denselben stetigen Testfunktionen getestet, wie der ursprüngliche Algorithmus $\text{ACO}_{\mathbb{R}}$ -VS. Diese sind Testfunktion zum Vergleich mit Algorithmen, die probabilistisches Lernen einsetzen in Tabelle 3 und Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Probleme adaptiert wurden, in Tabellen 4, 5 und 6.

Die Testfunktionen wurden aus verschiedene Quellen entnommen. Die Branin RCOS-Testfunktion stammt aus [27], die Goldstein and Price-Testfunktion aus [28] und die Griewangk-Testfunktion aus [18]. Die Griewangk-Testfunktion wurde auch in den Arbeiten von *Dorigo und Socha* [22] und *Socha* [21] benutzt, auf denen diese Arbeit aufbaut. Aber dort wurde eine andere Version der Griewangk-Testfunktion benutzt, als die Version in anderen Quellen, nämlich *Molga und Smutnicki* [18] und *Pohlheim* [19], die explizit Testfunktionen auflisten, um Algorithmen zu testen. Da in [22] und [21] keine Optima angegeben wurden, wurde in dieser Arbeit sich für die Version der Griewangk-Testfunktion aus [19] und [18] entscheiden. Die Testfunktionen Hartmann stammen aus [24] und [25] und die Shekel-Testfunktionen aus [26]. Alle anderen Testfunktionen stammen aus [22] und [21]. Bei der Hartmann(6,4)-Testfunktion haben sich die Quellen in ihrer Definition des Funktionsterms im Matriceintrag a_{15} widersprochen. In [22] und [21] wird $a_{15} = 1.5$ angegeben während in [25] angegeben wird, dass $a_{15} = 1.7$. Da nur in [25] das dazugehörige Optimum angegeben wird, wurde in dieser Arbeit sich für die Angabe aus dieser Quelle entschieden. Um zukünftige Forschung zu erleichtern, werden alle Testfunktionen in dieser Arbeit, genau wie in meiner Bachelorarbeit [16], nicht nur mit Name und Funktionsterm, sondern auch mit Definitionsbereich/Initialisierungsintervall, Optimum und Argument des Optimums angegeben.

In meiner Bachelorarbeit stimmten die von mir berechnete Optima für die Funktionen Hartmann(3,4), Hartmann(6,4), Shekel(4,5), Shekel(4,7) und Shekel(4,10) mit denen in der Literatur nicht überein. Ich konnte den Fehler finden. Ich habe die Funktionsgleichungen der Testprobleme in den Quellen falsch interpretiert, weswegen ich falsche Ergebnisse bekommen hatte. Für Shekel(4,k;k = 5,7,10) lag der Fehler in der Interpretationen der Gleichung 26. Ich habe die Gleichung so verstanden, wie es in der Gleichung 27 geschrieben ist. Aber der Term ist, wie in Gleichung 28 zu interpretieren.

$$f(x) = - \sum_{i=1}^k \left(\sum_{j=1}^4 (x_j - a_{ji})^2 + c_i \right)^{-1} \quad (26)$$

$$f(x) = - \sum_{i=1}^k \left(\sum_{j=1}^4 ((x_j - a_{ji})^2 + c_i) \right)^{-1} \quad (27)$$

$$f(x) = - \sum_{i=1}^k \left(\sum_{j=1}^4 ((x_j - a_{ji})^2) + c_i \right)^{-1} \quad (28)$$

Die Testfunktionen, welche in den Tabellen 3, 4, 5 und 6 gelistet sind, unterscheiden sich nicht nur darin, für welche Vergleiche sie herangezogen werden, sondern auch welches Abbruchkriterium beim Vergleich genutzt wird. Für Testfunktionen aus Tabelle 3 gilt das Abbruchkriterium $|f - f^*| < \varepsilon$, $\varepsilon = 10^{-10}$ und für Testfunktionen aus den Tabellen 4, 5 und 6 gilt das Abbruchkriterium $|f - f^*| < |\varepsilon_1 f| + \varepsilon_2$, $\varepsilon_1 = \varepsilon_2 = 10^{-4}$. Dabei ist ε_1 der relative Fehler und ε_2 der absolute Fehler. In beiden Abbruchkriterien steht f für den momentanen Funktionswert und f^* für das bekannte Optimum. Das zweite Abbruchkriterium wird in [22] und [21] als $|f - f^*| < \varepsilon_1 f + \varepsilon_2$, $\varepsilon_1 = \varepsilon_2 = 10^{-4}$ angegeben, aber es musste zu der jetzigen Form verändert werden, da sonst negative Optima nicht erreicht werden können. So wird z.B. das Optimum $f^* = -3.32237$ der Testfunktion Hartmann(6,4) nicht erreicht, da, für einen an das Optimum hinreichend nahen, Funktionswert $f = -3.32237 + \varepsilon_3$, mit $\varepsilon_3 > 10^{-4}$, gilt $|-3.32237 - (f + \varepsilon_3)| = |\varepsilon_3| = \varepsilon_3 > \varepsilon_1 \cdot (-3.32237) + \varepsilon_2$. Eine weitere Herausforderung des Abbruchkriteriums $|f - f^*| < \varepsilon_1 f + \varepsilon_2$, $\varepsilon_1 = \varepsilon_2 = 10^{-4}$, ist die Tatsache, dass 6 der in diesem Abschnitt vorgestellten 20 Testfunktionen Optima haben, die mehr als 4 Nachkommastellen besitzen, weswegen, bei der Benutzungen des zweiten Abbruchkriteriums, die Güte eines Algorithmus, der auf diesen 6 Testfunktionen getestet wird, besser ausgewertet wird, als sie ist. Denn bei dem zweiten Abbruchkriterium muss der Algorithmus z.B. für $f^* = -3.32237$ nur $f = -3.3223 \dots$ erreichen und gerade die letzte Nachkommastelle oder sogar Nachkommastellen, wenn es mehrere sind, können darüber entscheiden, wie viele Iterationen ein Algorithmus braucht, um das Optimum zu erreichen und ob das Optimum überhaupt gefunden wurde oder nur ein Wert, der nahe dem Optimum ist.

Ferner müssen wir noch auf das Initialisierungsintervall eingehen. Das Initialisierungsintervall ist das Intervall aus dem, bei der Initialisierung des Algorithmus, die ersten $k \in \mathbb{N}$ zufälligen Lösungen gezogen werden aus denen dann andere Lösungen konstruiert

werden. Dabei kann $k = 13$ sein, wie bei der Initialisierung des HSPPBO-Schema in Abschnitt 6, $k = 50$, wie bei der Initialisierung von $\text{ACO}_{\mathbb{R}}$ in Abschnitt 4.1 oder $k = 100$ bei der Initialisierung von $\text{ACO}_{\mathbb{R}}\text{-VS}$ im Abschnitt 4.3. Auch wenn das Initialisierungsintervall bei $\text{ACO}_{\mathbb{R}}$ und $\text{ACO}_{\mathbb{R}}\text{-MV}$ nach der Initialisierung nicht mehr verwendet wird, ist es trotzdem wichtig, da, wenn es schlecht gewählt wurde, die Initiallösungen so liegen, dass die Algorithmusleistung zum negativen oder zum positiven verzerrt wird. Wir müssen die Initialisierungsintervalle für die Testfunktionen aus den Tabellen 4, 4, 5 und 6 aber nicht näher betrachten, da mit den Initialisierungsintervallen für diese Testfunktionen sich schon in [22] und [21] auseinander gesetzt wurde. Deswegen können wir sie übernehmen. Eine offene Forschungsfrage ist, wie Optimierungsalgorithmen auf nicht zusammenhängende Definitionsbereiche reagieren.

Wenn es Nebenbedingungen/Constraints gibt, kann das vollständige Initialisierungsintervall sich vom Initialisierungsintervall, das durch Anwenden der Nebenbedingungen entsteht, stark unterscheiden, wie im Abschnitt 9.4.1. Ein solchen Intervall wird hier als gefiltert bezeichnet.

Genau wie in meiner Bachelorarbeit bezeichnen wir die Initialisierungsintervalle hier als Definitionsbereiche. Das liegt daran, dass die Funktionen, sowohl initial über diesen Intervallen definiert sind, als auch die Intervallgrenzen definieren, was gültige Lösungen sind. Der Suchraum für die Funktionen aus den Tabellen 3, 4, 5 und 6 ist zwar $\Sigma = \mathbb{R}^n$, wobei $n \in \mathbb{N}$ von Testfunktion abhängt, aber die Beschränkungen $\mathcal{B}$ sind die Definitionsbereiche, die für die verschiedenen Testfunktionen abhängig von der Testfunktion definiert sind, z.B. für die Testfunktion „Paraboloid“ gilt $\mathcal{B}_1 = \dots = \mathcal{B}_n = [-3, 7]$

Schließlich betrachten wir 6 ausgewählte Testfunktionen in den Abschnitten 9.2.1 bis 9.2.6 genauer. Diese Testfunktionen haben sich nicht nur durch besondere Herausforderungen, die sie an einen Optimierungsalgorithmus stellen, herausgetan, sondern auch dadurch, dass die Herausforderung, die bieten, singulär ist, sie also den Algorithmus auf nur eine Weise fordern und damit sich eignen den Algorithmus auf eine Stärke oder eine Schwäche zu testen und werden deswegen näher betrachtet.

Tabelle 3: Testfunktionen zum Vergleich mit Algorithmen, die probabilistisches Lernen einsetzen

Funktion, Definitionsbereich, Optimum	Formel
Cigar, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$ 10^{10} bei mehreren	$f(x) = x_1^2 + 10^4 \sum_{i=2}^n x_i^2$
Ellipsoid, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{i=1}^n \left(100^{\frac{i-1}{n-1}} x_i \right)^2$
Paraboloid, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$ 10^{10} bei $x_1 = 10^{10}$	$f(x) = \sum_{i=1}^n x_i^2$
Rosenbrock, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [1, 1, 1, \dots]$	$f(x) = \sum_{i=1}^{n-1} 100(x_i^2 - x_{i+1})^2 + (x_i - 1)^2$
Tablet, $\vec{x} \in [-3, 7]^n, n = 10$, 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = 10^4 x_1^2 + \sum_{i=2}^n x_i^2$

Tabelle 4: erster Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Probleme adaptiert wurden

Funktion, Definitionsbereich, Optimum	Formel
B_2 , $\vec{x} \in [-100, 100]^n, n = 2$, 0 bei $\vec{x} = [0, 0]$	$f(x) = x_1^2 + 2x_2^2 - \frac{3}{10}\cos(3\pi x_1) - \frac{2}{5}\cos(4\pi x_2) + 7/10$
Branin RCOS, $\vec{x} \in [-5, 15]^n, n = 2$, 0.397887 bei $\vec{x} = [-\pi, 12.275]$ od. $x = [\pi, 2.275]$ od. $x = [9.42478, 2.475]$	$f(x) = (x_2 - \frac{5}{4\pi}x_1^2 + \frac{5}{\pi}x_1 - 6)^2 + 10(1 - \frac{1}{8\pi})\cos(x_1) + 10$
Easom, $\vec{x} \in [-100, 100]^n, n = 2$, 0 bei $\vec{x} = [\pi \cdot k + \pi/2]$, $\pi \cdot k + \pi/2], k \in \mathbb{Z}$	$f(x) = -\cos(x_1) \cos(x_2) e^{-(x_1 - \pi)^2 - (x_2 - \pi)^2}$
De Jong, $\vec{x} \in [-5.12, 5.12]^n, n = 3$, 0 bei $\vec{x} = [0, 0, 0]$	$f(x) = x_1^2 + x_2^2 + x_3^2$
Goldstein, $\vec{x} \in [-2, 2]^n$ and Price, $n = 2$, 3 bei $\vec{x} = [0, 1]$	$f(x) = (1 + (x_1 + x_2 + 1)^2(19 - 14x_1 + 13x_1^2 - 14x_2 + 6x_1x_2 + x_2^2))(30 + (2x_1 - 3x_2)^2 \cdot (18 - 32x_1 + 12x_1^2 - 48x_2 - 36x_1x_2 - 27x_2^2))$
Griewangk, $\vec{x} \in [-600, 600]^n$, $n = 10$; 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{i=1}^n \frac{x_i^2}{4000} - \prod_{i=1}^n \cos(\frac{x_i}{\sqrt{i}}) - 1$
Martin and Gaddy, $\vec{x} \in [-20, 20]^n, n = 2$ 0 bei $\vec{x} = [5, 5]$	$f(x) = (x_1 - x_2)^2 + \left(\frac{x_1 + x_2 - 10}{3}\right)^2$
Paraboloid(6), $\vec{x} \in [-5.12, 5.12]^n$, $n=6$; 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{i=1}^n x_i^2$
Rosenbrock(2,5), $\vec{x} \in [-5, 5]^n$, $n = 2, 5$; 0 bei $\vec{x} = [1, 1, 1, \dots]$	$f(x) = \sum_{i=1}^{n-1} 100(x_i^2 - x_{i+1})^2 + (x_i - 1)^2$
Zakharov, $\vec{x} \in [-5, 10]^n$, $n = 2, 5$; 0 bei $\vec{x} = [0, 0, 0, \dots]$	$f(x) = \sum_{j=1}^n x_j^2 + \left(\sum_{j=1}^n \frac{jx_j^2}{2}\right)^2 + \left(\sum_{j=1}^n \frac{jx_j^2}{2}\right)^4$

Bei Funktionen, welche in mehreren Dimensionen getestet wurden, stehen die getesteten Dimensionen in Klammern hinter der Funktion. Dasselbe gilt für Funktionen, die sich in der getesteten Dimension von der Dimension aus Tabelle 3 unterscheiden.

Tabelle 5: zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Probleme adaptiert wurden

Funktion	Formel
Hartmann(3,4)	$f_{H_{3,4}}(x) = - \sum_{i=1}^4 c_i e^{-\sum_{j=1}^3 a_{ij}(x_j - p_{ij})^2}$ $a_{ij} = \begin{pmatrix} 3.0 & 10.0 & 30.0 \\ 0.1 & 10.0 & 35.0 \\ 3.0 & 10.0 & 30.0 \\ 0.1 & 10.0 & 35.0 \end{pmatrix} \quad c_i = \begin{pmatrix} 1.0 \\ 1.2 \\ 3.0 \\ 3.2 \end{pmatrix}$ $p = \begin{pmatrix} 0.3689 & 0.1170 & 0.2673 \\ 0.4699 & 0.4387 & 0.7470 \\ 0.1091 & 0.8732 & 0.5547 \\ 0.0381 & 0.5743 & 0.8828 \end{pmatrix}$
Hartmann(6,4)	$f_{H_{3,4}}(x) = - \sum_{i=1}^4 c_i e^{-\sum_{j=1}^6 a_{ij}(x_j - p_{ij})^2}$ $a_{ij} = \begin{pmatrix} 10.0 & 3.0 & 17.0 & 3.5 & 1.7 & 8.0 \\ 0.05 & 10.0 & 17.0 & 0.1 & 8.0 & 14.0 \\ 3.0 & 3.5 & 1.7 & 10.0 & 17.0 & 8.0 \\ 17.0 & 8.0 & 0.05 & 10.0 & 0.1 & 14.0 \end{pmatrix} \quad c_i = \begin{pmatrix} 1.0 \\ 1.2 \\ 3.0 \\ 3.2 \end{pmatrix}$ $p = \begin{pmatrix} 0.1312 & 0.1696 & 0.5569 & 0.0124 & 0.8283 & 0.5886 \\ 0.2329 & 0.4135 & 0.8307 & 0.3736 & 0.1004 & 0.9991 \\ 0.2348 & 0.1451 & 0.3522 & 0.2883 & 0.3047 & 0.6650 \\ 0.4047 & 0.8828 & 0.8732 & 0.5743 & 0.1091 & 0.0381 \end{pmatrix}$
Shekel(4,k; k = 5,7,10)	$f(x) = - \sum_{i=1}^k (\sum_{j=1}^4 (x_j - a_{ji})^2 + c_i)^{-1}$ $a_{ij} = \begin{pmatrix} 4.0 & 4.0 & 4.0 & 4.0 \\ 1.0 & 1.0 & 1.0 & 1.0 \\ 8.0 & 8.0 & 8.0 & 8.0 \\ 6.0 & 6.0 & 6.0 & 6.0 \\ 3.0 & 7.0 & 3.0 & 7.0 \\ 2.0 & 9.0 & 2.0 & 9.0 \\ 5.0 & 5.0 & 3.0 & 3.0 \\ 8.0 & 1.0 & 8.0 & 1.0 \\ 6.0 & 2.0 & 6.0 & 2.0 \\ 7.0 & 3.6 & 7.0 & 3.6 \end{pmatrix}^T \quad c_i = \begin{pmatrix} 0.1 \\ 0.2 \\ 0.2 \\ 0.4 \\ 0.4 \\ 0.6 \\ 0.3 \\ 0.7 \\ 0.5 \\ 0.5 \end{pmatrix}$

Tabelle 6: zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Probleme adaptiert wurden

Funktion, Definitionsbereich	Optimum
Hartmann(3,4), $\vec{x} \in [0, 1]^n, n = 3$	-3.86278 bei $\vec{x} = [0.114614, 0.555649, 0.852547]$
Hartmann(6,4), $\vec{x} \in [0, 1]^n, n = 6$	-3.32237 bei $\vec{x} = [0.20169, 0.150011, 0.476874,$ $0.275332, 0.311652, 0.6573]$
Shekel(4,5) $\vec{x}, \in [0, 1]^n, n = 4$	-10.1532 bei $\vec{x} = [4, 4, 4, 4]$
Shekel(4,7) $\vec{x}, \in [0, 1]^n, n = 4$	-10.4029 bei $\vec{x} = [4, 4, 4, 4]$
Shekel(4,10) $\vec{x}, \in [0, 1]^n, n = 4$	-10.5364 bei $\vec{x} = [4, 4, 4, 4]$

9.2.1. Testfunktion - Paraboloid

Die Paraboloid-Testfunktion ist unimodal und konvex. Sie ist die einfachste aller einfachen Testfunktion und somit insgesamt die einfachste Testfunktion. Sie dient sowohl als Demonstration, dass der Algorithmus, welcher getestet wird, ordnungsgemäß funktioniert, als auch zum Testen der Konvergenzgeschwindigkeit des Algorithmus. Konvergiert ein Algorithmus zu schnell, kann er ein globales Optimum überspringen, konvergiert er zu langsam, erreicht er das globale Optimum nicht in der vorgegebenen Menge an Ressourcen, wobei Ressourcen maximale Ausführungszeit oder maximale Anzahl an Funktionsausführungen sein können. Letzteres ist für die Testfunktion Rosenbrock eingetreten, als die Initialversion des ACO_ℝ-VS Algorithmus aus meiner Bachelorarbeit [16] versucht hat diese zu optimieren. Ferner haben *Merkle und Middendorf* [17] gezeigt, dass die Untersuchung einfacher Probleme ein lohnendes Unterfangen sein kann, da einfache Probleme ein unvorhergesehenes Verhalten des Algorithmus aufdecken können.

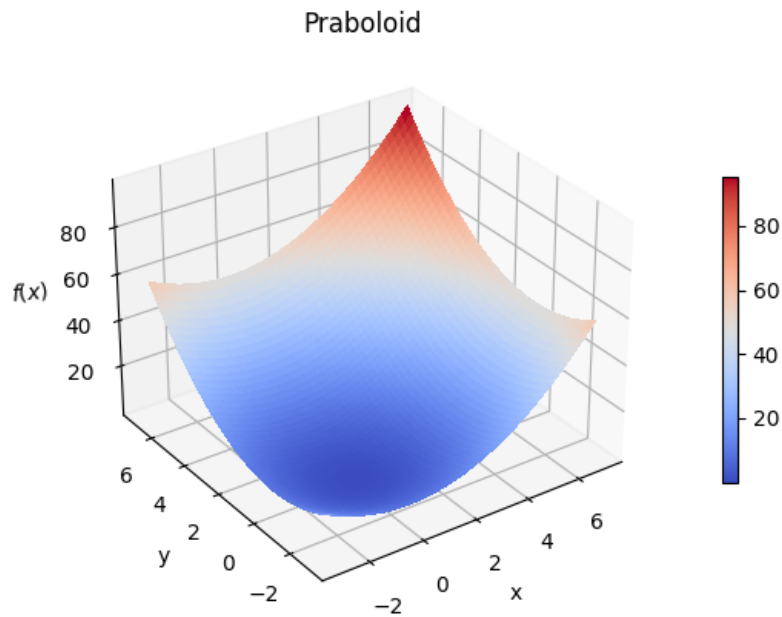


Abbildung 2: Paraboloid - Visualisierung in 3D

9.2.2. Testfunktion - Easom

Wie in [18] beschrieben, ist die Easom-Testfunktion deswegen interessant, weil die Umgebung um das globale Minimum klein ist im Verhältnis zur Größe des gesamten Suchraum. Der Suchraum ist das Quadrat $[-100, 100] \times [-100, 100]$ in der Abbildung 3 links, während das Optimum im Quadrat $[1, 5] \times [1, 5]$ sich befindet, in der Abbildung 3 rechts.

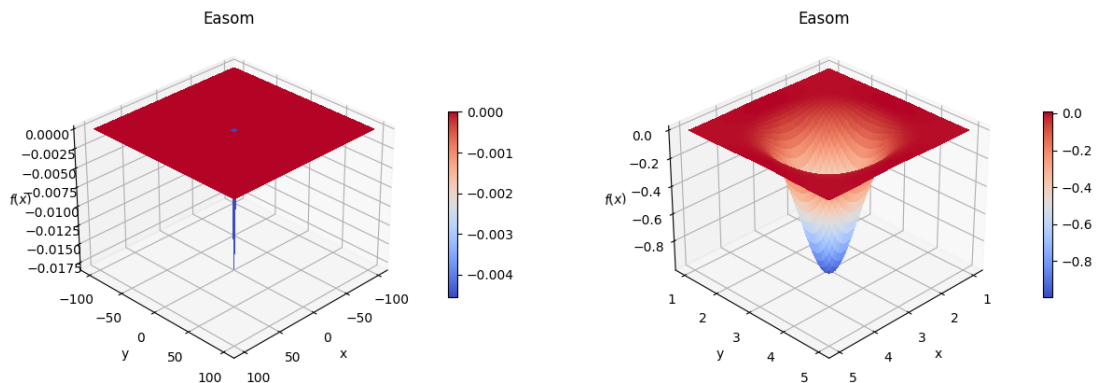


Abbildung 3: Easom-Testfunktion: Vergleich zwischen gesamtem Suchraum (links) und Umgebung um das Optimum (rechts)

9.2.3. Testfunktion - Griewangk

Wie in [18] beschrieben ist die Griewangk-Testfunktion multimodal und hat damit viele lokale Optima, was die Suche nach dem globalen für Algorithmen erschwert. Ferner sind die lokalen Minima zwar gleich verteilt aber weit gestreut, was die Suche nach dem globalen Optima weiter verkompliziert. Die Griewangk-Testfunktion wurde auch deswegen gewählt, da, wie in Abbildung 4 zusehen, sich das Aussehen der Funktion in Abhängigkeit vom Betrachtungsintervall ändert. Im Intervall $[-600, 600]^n$, $n = 2$, was die 2-dimensionale Version des Initialisierungsintervall $[-600, 600]^n$, $n = 10$ ist, welches für die Optimierung verwendet wird, muss der Algorithmus schnell konvergieren, um nicht viele Iterationen mit schlechten Werten zu verbrauchen. Im Intervall $[-10, 10]^n$, $n = 2$, muss der Algorithmus Variabilität zeigen, um die große Menge an lokalen Minima zu meiden. Und schließlich im Intervall $[-4, 4]^n$, $n = 2$ muss der Algorithmus eine aperiodische, zerklüftete Landschaft manövrierern, um das globale Optimum zu finden. Deswegen eignet sich die Griewangk-Testfunktion besonders gut, um das Können eines Algorithmus zu testen.

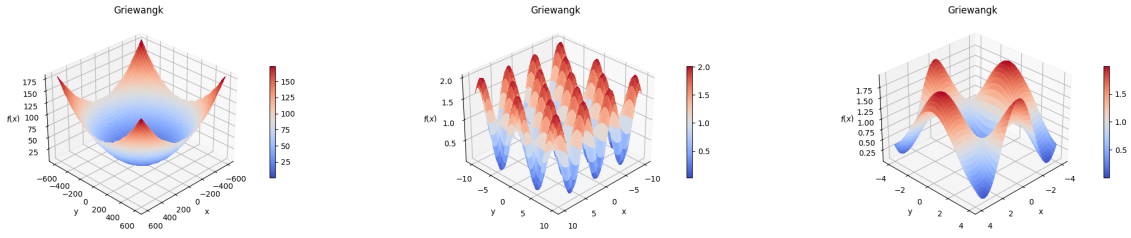


Abbildung 4: Verschiedene Auflösungen der Griewangk-Testfunktion

9.2.4. Testfunktion - Rosenbrock

Wie in [18] beschrieben besteht die Rosenbrock-Testfunktion dadurch, dass ihr Optimum in einem langen, engen Tal liegt. Es ist leicht das Tal zu finden aber es ist sehr schwer zum Optimum zu konvergieren. Die Rosenbrock-Testfunktion ist interessant, da sie sehr flach nahe dem Optimum ist und die Konvergenz langsam vorangeht. Die Rosenbrock-Testfunktion wurde auch aus dem Grund gewählt, weil in meiner Bachelorarbeit [16], unter den Testfunktionen zum Vergleich mit Algorithmen, die probabilistisches Lernen einsetzen, den Testfunktionen aus Tabelle 3, es die einzige Testfunktion war, bei welcher der $ACO_{\mathbb{R}}$ -very-simple nicht der beste Algorithmus war oder zumindest sehr gut. Bei dieser Testfunktion konnte $ACO_{\mathbb{R}}$ -VS das Optimum nicht finden. Deswegen ist diese Testfunktion besonders interessant, um die Variationen von $ACO_{\mathbb{R}}$ -VS zu testen.

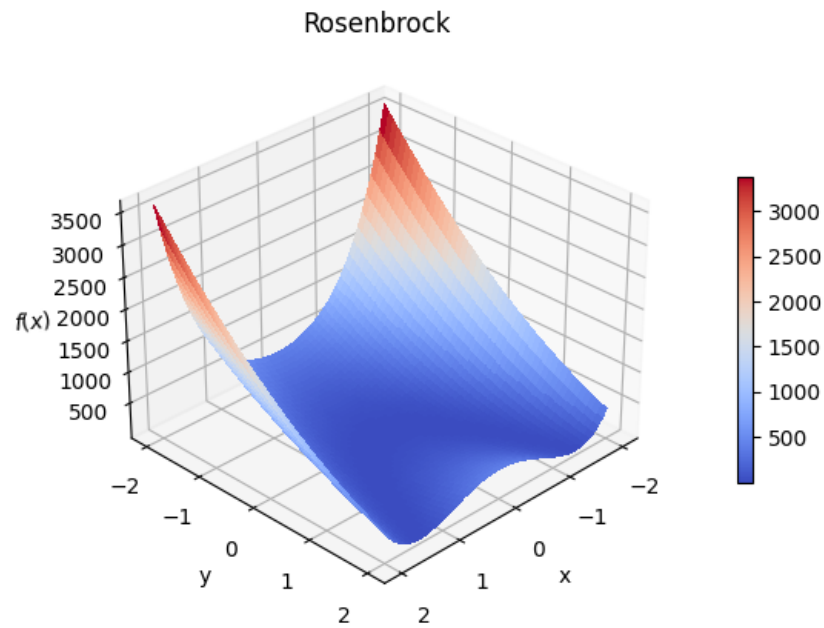


Abbildung 5: Rosenbrock - Visualisierung in 3D

9.2.5. Testfunktion - Hartmann

Die Familie der Hartmann-Testfunktionen ist eine der beiden Familien der komplexen Testfunktionen, die in dieser Arbeit betrachtet werden. Diese Familie zeichnet sich dadurch aus dass sie die leichtere der beiden komplexen Testfunktionfamilien ist. Sie ist multimodal mit 4 lokalen Minima im Fall von Hartmann(3,4), und respektive 6 lokalen Minima im Fall von Hartmann(6,4).

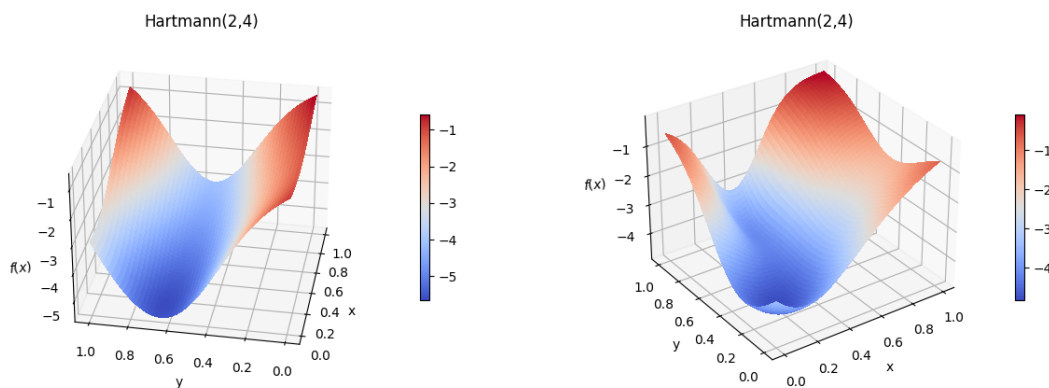


Abbildung 6: Hartmann-Testfunktionen: Hartmann(2,4) (links) und Hartmann(2,6) (rechts)

9.2.6. Testfunktion - Shekel

Die Familie der Shekel-Testfunktionen ist die schwerere der beiden komplexen Testfunktionfamilien, die in dieser Arbeit betrachtet werden. Sie ist multimodal und hat 5 lokale Minima im Fall von Shekel(4,5), 7 lokale Minima im Fall von Shekel(4,7) und respektive 10 lokale Minima im Fall von Shekel(4,10).

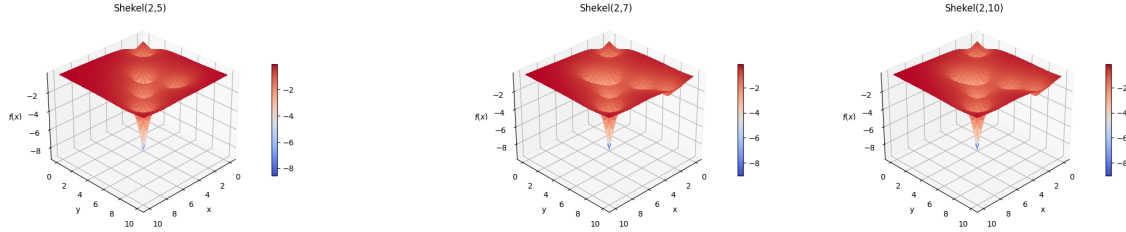


Abbildung 7: Shekel-Testfunktionen: Shekel(2,5) (links), Shekel(2,7) (mittig) und Shekel(2,10) (rechts)

9.3. Diskrete Testfunktionen

9.3.1. Ordinale Testfunktionen mit gleichverteiltem Definitionsbereich

Die einfachste Art diskrete/ordinale Testfunktionen zu erhalten, ist die Diskretisierung von stetigen Testfunktionen. Dabei nimmt man $m \in \mathbb{N}$ Punkte aus dem Definitionsbereich einer stetigen Funktion und bildet aus diesen Punkten einen neuen diskreten Definitionsbereich. Zum Beispiel ist die Funktion „Paraboloid“ aus Tabelle 3 über dem stetigen Definitionsbereich $D = [-3, 7]^n, n = 10$ definiert. Wir nehmen diesen Definitionsbereich und zerlegen ihn in $m = 11$ gleichverteilte diskrete Punkte. Dies ergibt $D = [-3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7]^n, n = 10$. Analog gewinnen wir anderen diskreten Testfunktionen. Wir nehmen die stetigen Testfunktionen aus den Tabellen 3 und 4 mit $m \in \{11, 21, 51\}$. Damit erhalten wir für jede stetige Funktion 3 ordinale/diskrete Testfunktionen. Die Definitionen der Funktionsterme, Optima und Argumente der Optima bleiben gleich, weswegen hier auf eine Doppelung der Tabellen 3 und 4 verzichtet wird. Wie im Abschnitt 4.1 hat das Initialisierungsintervall großen Einfluss auf das Verhalten des Algorithmus aber da wir eine gleichverteilte Diskretisierung genommen haben, um die neuen Definitionsbereiche zu erlangen, ist zu erwarten, dass sich das Lösungsverhalten für die ordinalen/diskreten Testfunktionen nicht signifikant von denen in den Tabellen 3 und 4 unterscheiden wird.

9.3.2. Ordinale Testfunktionen mit asymmetrischem Definitionsbereich

Wie in Abschnitt 4.1 geschildert, hat das Initialisierungsintervall großen Einfluss auf das Verhalten des Algorithmus. Zusätzlich, wie in [17] beschrieben, kann das Betrachten von einfachen Testfunktionen zu unerwarteten Ergebnissen führen. Deswegen definieren

wir in diesem Abschnitt explizit diskrete Funktionen, die einfach sind aber einen nicht-Standarddefinitionsbereich haben. Wir wählen, wie im Abschnitt 9.3.1, $m \in \{11, 21, 51\}$ aber diesmal ist der Definitionsbereich asymmetrisch verteilt mit 3 Clustern. Zum Beispiel transformiert dies den stetigen Definitionsbereich der Testfunktion „Paraboloid“ von $D = [-3, 7]^{10}$ zu $D = [-3, -2.8, -2.6, -0.4, -0.2, 0.0, 0.2, 0.4, 6.6, 6.8, 7.0]^{10}$. Im JSON-Format wird dies wie im Code-Snipped 2 angezeigt.

Code-Snipped 2: Paraboloid 2Dim ordinal asymmetrischer Definitionsbereich

```

1 {
2     "name": "Paraboloid_2_11_Ord_Asymm",
3     "comment": "artificial ordinal variable problem",
4     "type": "ordinal variable problems",
5     "continuous_dimension": 0,
6     "discrete_dimension": 2,
7     "dimension": 2,
8     "optimum_argument": [[0.0], [0.0]],
9     "optimum": [0.0],
10
11     "variable_boundaries": {
12         "natural_numbers": [],
13         "discrete": [[-3, 7]],
14         "continuous": [],
15         "ordinal": [],
16         "categorical": []
17     },
18
19     "discrete_values": [-3, -2.8, -2.6,
20                         -0.4, -0.2, 0.0, 0.2, 0.4,
21                         6.6, 6.8, 7.0],
22
23     "magnitudes_of_categorical_variables": []
24 }
```

9.4. Testfunktionen mit gemischten Variablen

9.4.1. Coil Spring Design Problem (CSD)

Das Coil Spring Design Problem (CSD) kann auf verschiedene Arten definiert werden, wie in [11], [14], oder []. In dieser Arbeit verwenden wir die Definition aus [14]. Im CSD Problem, wie es in [14] definiert wird, wird so nach dem Außendurchmesser D , dem Drahtdurchmesser d und der Windungszahl N einer Schraubenfeder (Coil Spring) gesucht, dass der Materialbedarf für die Schraubenfeder minimiert wird, während gleichzeitig die Nebenbedingungen (Constraints) erfüllt werden.

Da in *Sandgren* [20] das CSD Problem initial in imperialen Maßeinheiten definiert wurde, verwendeten alle Folgestudien ebenfalls imperiale Maßeinheiten, um die Ergebnisse vergleichen zu können. Deswegen werden auch in dieser Arbeit hier imperiale Maßeinheiten verwendet.

Das Problem zählt zu Problemen mit gemischten Variablen, da D eine stetige und d eine ordinale Variable ist. N ist eine Variable, die nur natürliche Zahlen als Werte annehmen kann.

Mögliche Werte, die der Drahtdurchmesser d annehmen kann, sind in der Tabelle 7 aufgelistet.

Tabelle 7: Standarddrahtdurchmesser für die Schraubenfeder angegeben in Zoll (inch)

0.0090	0.0095	0.0104	0.0118	0.0128	0.0132
0.0140	0.0150	0.0162	0.0173	0.0180	0.0200
0.0140	0.0150	0.0162	0.0173	0.0180	0.0200
0.0470	0.0540	0.0630	0.0720	0.0800	0.0920
0.1050	0.1200	0.1350	0.1480	0.1620	0.1770
0.1920	0.2070	0.2250	0.2440	0.2630	0.2830
0.3070	0.3310	0.3620	0.3940	0.4375	0.5000

Weiter werden in [14] Intervallgrenzen definiert, die in der Tabelle 8 aufgeführt werden.

Tabelle 8: Intervallgrenzen

maximale Arbeitsbelastung	$F_{max} = 1000.0$ Pfund (lb)
zulässige maximale Scherspannung	$S = 189000.0$ Pfund pro Quadratzoll (psi)
maximale freie Länge	$l_{max} = 14.0$ Zoll (inch)
Minstdrahtdurchmesser	$d_{min} = 0.2$ Zoll (inch)
maximaler Außendurchmesser der Feder	$D_{max} = 3.0$ Zoll (inch)
Vorspannkompressionskraft	$F_p = 300.0$ Pfund (lb)
zulässige maximale Durchbiegung unter Vorspannung	$\sigma_{pm} = 6.0$ Zoll (in)
Auslenkung von der Vorlastposition zur Maximallastposition	$\sigma_w = 1.25$ Zoll (in)
Schubmodul des Materials	$G = 11.5 \cdot 10^6$

Die Nebenbedingen umfassen primäre und sekundäre Variablen. Die primären Variablen sind D , d und N . Die sekundären Variablen sind C_f , K , σ_p und l_f . Die sekundären Va-

riablen ergeben sich aus den primären Variablen und den Intervallgrenzen und werden in den Gleichungen 29 bis 32 definiert. Die Definitionsbereiche der primären Variablen können der Tabelle 9 entnommen werden.

Tabelle 9: Definitionsbereiche

D	$\{3d, 3.0\}$
d	$\{d_{min}, 0.5000\}$
N	$\{1, 70\}$

$$C_f = \frac{4\frac{D}{d} - 1}{4\frac{D}{d} - 4} + \frac{0.615d}{D} \quad (29)$$

$$K = \frac{Gd^4}{8ND^3} \quad (30)$$

$$\sigma_p = \frac{F_p}{K} \quad (31)$$

$$l_f = \frac{F_{max}}{K} + 1.05(N + 2)d \quad (32)$$

Ferner unterliegen die Variablen im CSD 8 Constrains c_i , $i \in \{1, \dots, 8\}$. Diese sind in Tabelle 10 dargelegt.

Tabelle 10: Constrains

1	$c_1 = \frac{8C_f F_{max} D}{\pi d^3} - S \leq 0$
2	$c_2 = l_f - l_{max} \leq 0$
3	$c_3 = d_{min} - d \leq 0$
4	$c_4 = D - D_{max} \leq 0$
5	$c_5 = 3.0 - \frac{D}{d}$
6	$c_6 = \sigma_p - \sigma_{pm} \leq 0$
7	$c_7 = \sigma_p + \frac{F_{max} - F_p}{K} + 1.05(N + 2)d - l_f \leq 0$
8	$c_8 = \sigma_w - \frac{F_{max} - F_p}{K} \leq 0$

Der Materialbedarf und damit die Zielfunktion wird mit Hilfe der Formel ?? berechnet.

$$f(N, D, d) = \frac{\pi^2 D d^2 (N + 2)}{4} \quad (33)$$

9.4.1.1. Vollständiger Raum als Initialisierungsintervall

Wie bereits im Abschnitt 4.1 beschrieben, hat das Initialisierungsintervall großen Einfluss auf den Erfolg des Algorithmus. So können für das CSD Anfangslösungen aus dem gesamten Raum $\{3 \cdot d_{min}, 3.0\} \times \{d_{min}, 0.5000\} \times \{1, 70\}$ oder dem gefilterten Raum gezogen werden, was für das CSD einen Unterschied macht. Nimmt man den gesamten Raum als Grundlage, so findet ACO_ℝ-VS das Optimum nicht oder nur nach mehreren tausend Iterationen, wie in der Abbildung ?? zu sehen.

Nimmt man den gesamten Raum zum ziehen der Initiallösungen, so ändert sich die Zielfunktion von der Formel 33 zu der Formel 34, wie in [14] definiert.

$$\hat{f} = f \prod_{i=1}^8 \hat{c}_i^3 \quad (34)$$

Dabei werden die $\hat{c}_i$'s in der Formel 35 definiert.

$$\hat{c}_i = \begin{cases} 1 + w_i c_i, & \text{wenn } c_i > 0 \\ 1, & \text{sonst} \end{cases} \quad (35)$$

Die Gewichte w_i , $i \in \{1, \dots, 8\}$ wurden aus [14] entnommen und befinden sich in der Tabelle 11.

Tabelle 11: Gewichte

1	$w_1 = 10^{-5}$
2	$w_2 = 1$
3	$w_3 = 10^2$
4	$w_4 = 1$
5	$w_5 = 10^2$
6	$w_6 = 1$
7	$w_7 = 10^2$
8	$w_8 = 10^2$

9.4.1.2. Gefilterter Raum als Initialisierungsintervall

Wenn der gefilterte Raum genommen wird, konvergiert der Algorithmus sehr viel schneller, findet aber auch die selben lokalen Minima, anstatt des globalen Minima. Ferner werden mehrere tausend Iterationen für die Initialisierung gebraucht, da Punkte, welche die Constrains verletzen, gefiltert werden. Benutzt man den gefilterten Raum, kann die Formel 33 ohne Veränderungen beibehalten werden. Abbildung ?? zeigt, wie viele Iterationen bei gefiltertem Raum, für die Initialisierung und für die Lösungssuche selbst, notwendig waren und Abbildung 8 zeigt, wie stark sich der Initialisierungsraum verkleinert und damit die Lösungssuche erleichtert wird, wenn der Initialisierungsraum vor dem Ausführen des Algorithmus nach gültigen Lösungen gefiltert wird. Wie zu sehen verkleinert sich der Initialisierungsraum um mehr als das achtfache.

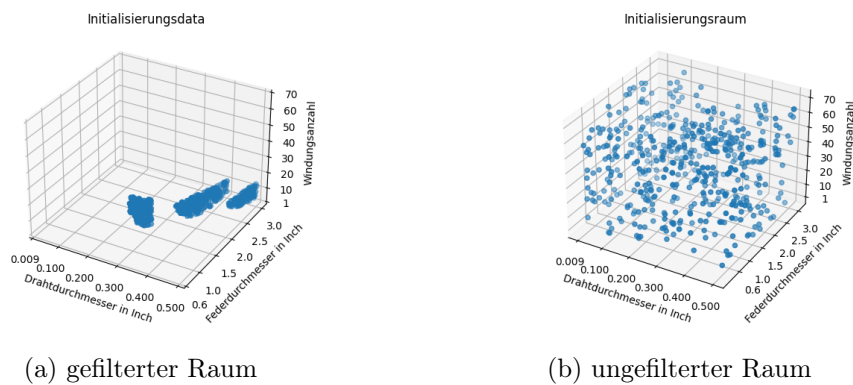


Abbildung 8: gefilterter und ungefilterter Suchraum für das CSD-Problem

Um zu zeigen, wie stark der Initialisierungsraum sich verkleinert, werden die beiden Initialisierungsräume wie in Abbildung 8

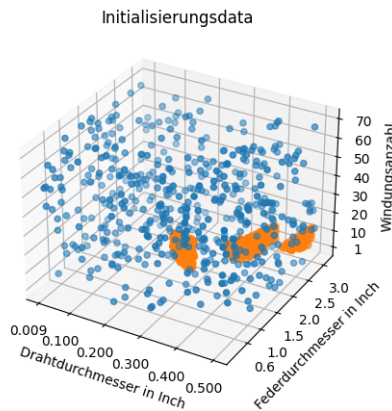


Abbildung 9: gefilterter und ungefilterter Raum

9.4.2. Thermal Insulation System Design Problem (TISD)

Das klassische nicht-konstruierte Testproblem, mit kategorischen Variablen, ist das Thermal Insulation System Design Problem (TISD). In der Literatur, wie in [14] beschrieben

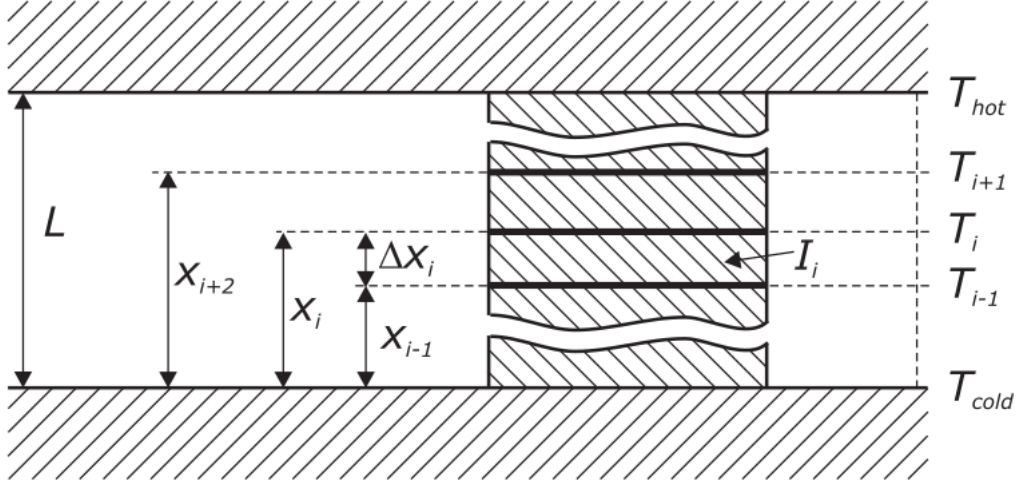


Figure 6.7: Schematic of the thermal insulation system.

Abbildung 10: Schematische Zeichnung des TISD, wie dargestellt in [14]

9.4.3. Traveling Spice Salesperson Problem (TSSP)

10. Herausforderung

11. Einfluss der Parameter

Bevor wir $ACO_{\mathbb{R}}$, $ACO_{\mathbb{R}}\text{-simple}$ und $ACO_{\mathbb{R}}\text{-very-simple}$ mit einander vergleichen, betrachten wir den Einfluss der Parameter in $ACO_{\mathbb{R}}\text{-simple}$ und $ACO_{\mathbb{R}}\text{-very-simple}$. Die dieser Arbeit beiliegenden Algorithmen sind von zwei bzw. drei Parametern beeinflusst. Im Fall $ACO_{\mathbb{R}}\text{-simple}$ sind es q , k und ξ und im Fall von $ACO_{\mathbb{R}}\text{-very-simple}$ sind es k und ξ , wie respektive in den Gleichungen (18) und (20) bzw. der Gleichung (??) zu sehen ist – zumindest nach Konstruktion. Tests haben jedoch gezeigt, dass es vier bzw. drei Parameter sind, nämlich, dass die Anzahl der Ameisen auch eine wesentliche Rolle spielt.

Der Einfluss der Parameter wird außerdem geprüft, da es sich bei $ACO_{\mathbb{R}}\text{-simple}$ und $ACO_{\mathbb{R}}\text{-very-simple}$ lediglich um Intervallauswertungsmethoden handelt und hier der Einfluss der Parameter anders sein kann, als bei $ACO_{\mathbb{R}}$, wir also zuerst überprüfen, ob er gleich ist oder nicht. Es besteht nämlich die Möglichkeit, dass aufgrund der zum Teil radikalen Einfachheit, vor allem von $ACO_{\mathbb{R}}\text{-very-simple}$, gegenüber $ACO_{\mathbb{R}}$, der Einfluss

der Parameter, im Vergleich zu $\text{ACO}_{\mathbb{R}}$, einen anderen Charakter annimmt.

Es wird jedoch davon ausgegangen, da sich alle Methoden doch ähneln, dass der Einfluss vergleichbar oder sogar der gleiche ist.

Abschließend ist noch festzustellen, dass durch die Betrachtung der Einflüsse der Parameter auf $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple, eine Art Parameteroptimierung bzw. Parameter Tuning durchgeführt wird. Das ist zwar nicht das Ziel der Betrachtung, aber wenn wir die Einflüsse verschiedener Parameterwerte auf die Leistungsfähigkeit der Algorithmen untersuchen, untersuchen wir auch gleichzeitig die Güte der Wahl der Parameterwerte. Es wird jedoch an dieser Stelle nochmal betont, dass das Ziel der Betrachtung, der Einflüsse der Parameter, die Untersuchung der Ähnlichkeit der Varianten zur Lösungserzeugung ist.

11.1. Einfluss der Anzahl der Ameisen

Normalerweise wird bei einem Ameisenalgorithmus die Anzahl der Ameisen klein gehalten, aber sie erreicht niemals eins. $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple arbeiten jedoch, wie in der Abbildung 11, Abbildung 16, Abbildung 17 und Abbildung 18 zu sehen, am besten mit nur einer einzigen Ameise, also wenn $m=1$.

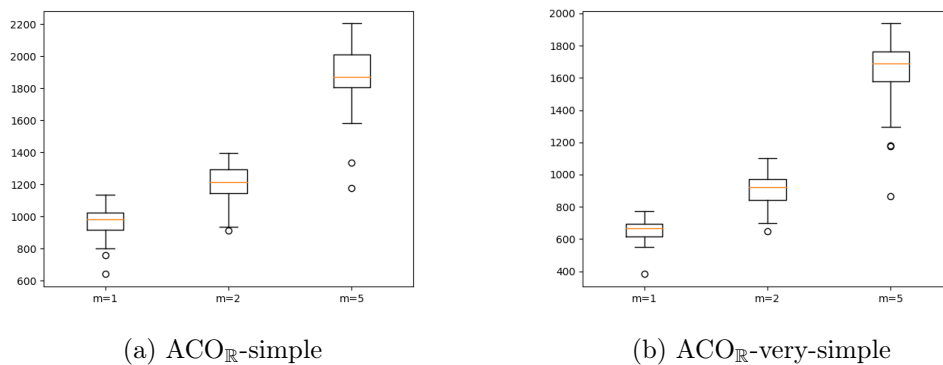


Abbildung 11: Ellipsoid Vergleich m

Da in $\text{ACO}_{\mathbb{R}}$, m kein Parameter war und dort somit nicht diskutiert wurde, können wir leider keine Aussage darüber treffen, ob die Algorithmen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple in ihrem Verhalten bzgl. des Parameters m ähnlich zu $\text{ACO}_{\mathbb{R}}$ sind.

11.2. Einfluss des Parameters q

Wie in $\text{ACO}_{\mathbb{R}}$, erwarten wir in $\text{ACO}_{\mathbb{R}}$ -simple, dass q die Lokalität der Suche steuert - je kleiner q , desto mehr werden die besseren Lösungen zum Generieren neuer Lösungen bevorzugt, desto schneller konvergiert der Algorithmus. Dies ist auch der Fall, was anhand von Abbildung 12, Abbildung 19, Abbildung 20 und Abbildung 21 zu sehen ist.

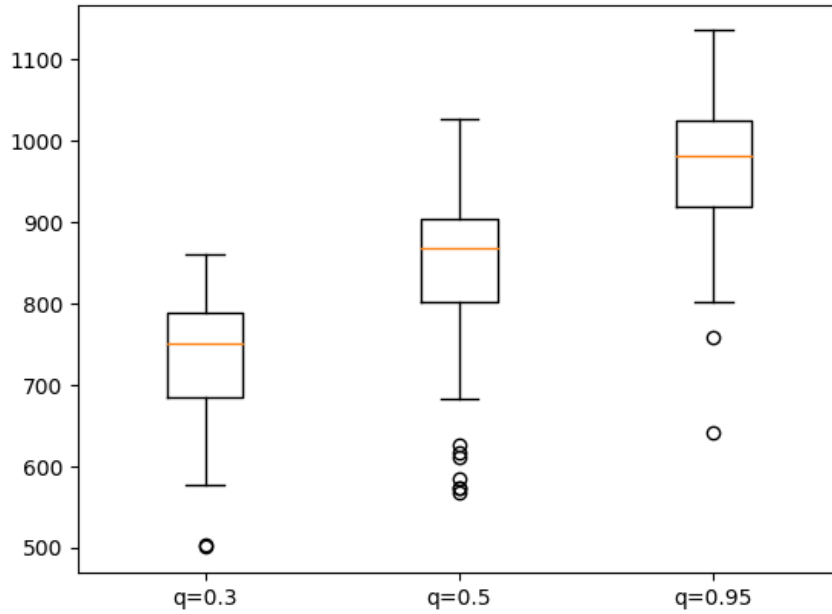


Abbildung 12: Ellipsoid Vergleich q

Der Algorithmus ist also in seinem Verhalten bzgl. des Parameters q ähnlich zu $\text{ACO}_{\mathbb{R}}$.

Für $\text{ACO}_{\mathbb{R}}$ -very-simple gibt es keine Abbildungen bzgl. des Einflusses des Parameters q , da dieser Parameter bei $\text{ACO}_{\mathbb{R}}$ -very-simple entfällt. Jedoch stellen wir fest, dass der Algorithmus sehr schnell konvergiert, viel schneller als $\text{ACO}_{\mathbb{R}}$ -simple, wie in Tabelle 14 zu sehen. Ferner können wir die Abwesenheit des Parameters q mit einem hinreichend kleinen q gleichstellen, da, wenn q sehr klein ist, nur die erste, also beste Lösung als Ausgangspunkt für die folgende Lösungserzeugung dient und $\text{ACO}_{\mathbb{R}}$ -simple selbst viel schneller konvergiert, als beim Standardwert $q=0.95$. Wir interpretieren also die schnelle Konvergenz von $\text{ACO}_{\mathbb{R}}$ -very-simple als den Einfluss eines hinreichend kleinen q .

Beide Algorithmen, $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple, sind also in ihrem Verhalten bzgl. des Parameters q ähnlich zu $\text{ACO}_{\mathbb{R}}$.

11.3. Einfluss des Parameters k

Wie zu erwarten war, funktionieren beide Algorithmen, $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple, mit größeren k besser. Es wurde für alle späteren Durchläufe $k=100$ gesetzt.

Da in $\text{ACO}_{\mathbb{R}}$, k kein Parameter war und dort somit nicht diskutiert wurde, können wir leider keine Aussage darüber treffen, ob die Algorithmen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple in ihrem Verhalten bzgl. des Parameters k ähnlich zu $\text{ACO}_{\mathbb{R}}$ sind.

11.4. Einfluss des Parameters ξ

In $\text{ACO}_{\mathbb{R}}$ hat der Parameter ξ eine ähnliche Wirkung wie die Verdunstungsrate des Pheromons. Je größer ξ , desto langsamer konvergiert der Algorithmus. In $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple vergrößert ξ das Suchintervall direkt, wodurch es einen noch stärkeren Einfluss auf die Konvergenz des Algorithmus haben sollte, jedoch stellt sich heraus, dass ξ nur einen geringen Einfluss, bei $\text{ACO}_{\mathbb{R}}$ -simple, auf die Testfunktion „Ellipsoid“ hat, wie Abbildung 13 zeigt.

Für die Testfunktionen „Cigar“, „Paraboloid“ und „Tablet“ wurde das Ziel jedoch nur bei $\xi = 0.95$ und $\xi = 1.5$ erreicht. Was wiederum einen sehr starken Einfluss nahelegt.

Diese Beobachtung lässt sich dadurch erklären, dass „Ellipsoid“ die Testfunktion ist, für welche $\text{ACO}_{\mathbb{R}}$ -simple mit großem Abstand die besten Werte liefert, die Leistung des Algorithmus also für diese Testfunktion vom ausgewählten Parameter wenig beeinflusst wird. Somit betrachten wir, wie der Algorithmus sich bei anderen Testfunktionen verhält.

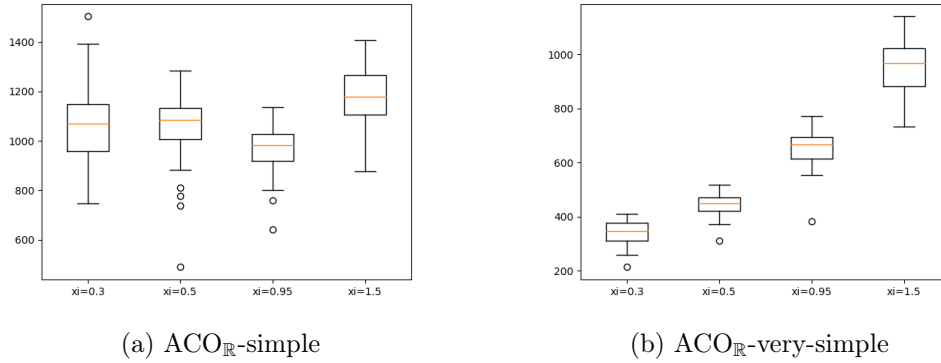


Abbildung 13: Ellipsoid Vergleich ξ

Wenn wir also das Verhalten von $\text{ACO}_{\mathbb{R}}$ -simple bei der Testfunktion „Ellipsoid“ als Eigenheit des Algorithmus ausklammern und uns auf die restlichen Ergebnisse konzentrieren, stellen wir fest, dass beide Algorithmen, sowohl $\text{ACO}_{\mathbb{R}}$ -simple als auch $\text{ACO}_{\mathbb{R}}$ -very-simple, bei kleinerem ξ nicht konvergieren und bei größerem ξ langsamer konvergieren, wie Abbildung 22, Abbildung 23 und Abbildung 24 zeigen. Beide Algorithmen sind also in ihrem Verhalten bzgl. des Parameters ξ ähnlich zu $\text{ACO}_{\mathbb{R}}$.

11.5. Auswertung der Einflüsse der Parameter

Wir stellen fest, dass die Einflüsse der Parameter q und ξ in $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple ähnlich sind, wie die Einflüsse dieser Parameter in $\text{ACO}_{\mathbb{R}}$, wie wir es erwartet haben.

Die Einflüsse sind jedoch nur gleich für den Parameter q . Für den Parameter ξ sind sie verschieden, wie Abbildungen 13, 22, 23 und 24 zeigen. Denn wie wir den Abbildungen entnehmen können, konvergieren weder $\text{ACO}_{\mathbb{R}}$ -simple noch $\text{ACO}_{\mathbb{R}}$ -very-simple für kleine ξ , während wir davon ausgehen, dass dies bei $\text{ACO}_{\mathbb{R}}$ nicht der Fall ist, da in *Socha und Dorigo* [22] erwähnt wurde, dass $\text{ACO}_{\mathbb{R}}$ für größere ξ langsamer konvergiert. Wir gehen also davon aus, dass $\text{ACO}_{\mathbb{R}}$ für kleinere ξ nur schneller konvergiert und dies einfach nicht genauer untersucht wurde.

Zum Schluss stellen wir noch fest, dass der Parameter ξ erstaunlicherweise optimal gewählt wurde, der Parameter q sollte allerdings möglichst klein sein. Es war zwar nicht das Ziel der Betrachtung eine Parameteroptimierung durchzuführen, jedoch ist dies ein Nebenprodukt, dass bei einer Diskussion der Parameter unweigerlich entsteht.

Da wir jedoch ausdrücklich die Algorithmen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple in ihrer original konzipierten Fassung untersuchen wollen und zudem der einzige Unterschied zwischen den beiden die Größe des Parameters q ist, bei $\text{ACO}_{\mathbb{R}}$ -very-simple können wir sagen, q ist sehr klein, müssen wir das Nebenprodukt der Untersuchung der Parametereinflüsse ignorieren. Denn wenn wir es mit einarbeiten, stellen wir fest, dass ein kleineres q besser ist, wir $\text{ACO}_{\mathbb{R}}$ -very-simple also zweimal untersuchen und hier gerade die Unterschiede zwischen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple von Interesse sind.

12. Vergleich zweier Intervallberechnungen für $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple

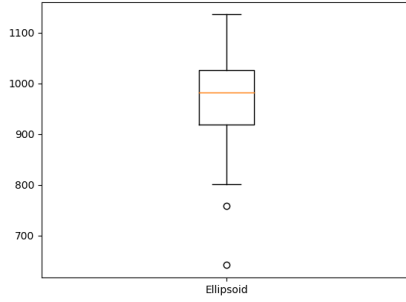
Nun gilt es, den Begriff der Distanzberechnung zu untersuchen. Da es Unstimmigkeiten bzgl. der Benennung der Intervallberechnung gab, werden diese im folgenden geklärt. In *Socha und Dorigo* [22] wurde

$$\delta_d = \xi \sum_{j=1}^k \frac{|s_j^d - s_i^d|}{k-1}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\}, \quad (36)$$

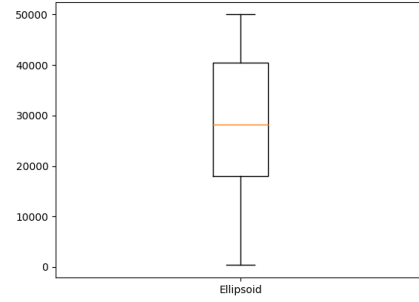
als Standardabweichung bezeichnet. Dies ist jedoch als lineare Abweichung bekannt. Wir untersuchen also im Folgenden, ob es bessere Ergebnisse liefert, statt der linearen Abweichung, Gleichung (36), die Standardabweichung, Gleichung (37) zu nehmen.

$$\delta_d = \xi \sqrt{\sum_{j=1}^k \frac{|s_j^d - s_i^d|}{k-1}}, \quad j \in \{1, \dots, k\}, \quad d \in \{1, \dots, n\}, \quad (37)$$

Es stellt sich heraus, dass die Intervalle, welche die lineare Abweichung liefert, also die Berechnung anhand Gleichung (36), die besseren sind, wie Abbildung 14 und Abbildung 15 verdeutlichen.

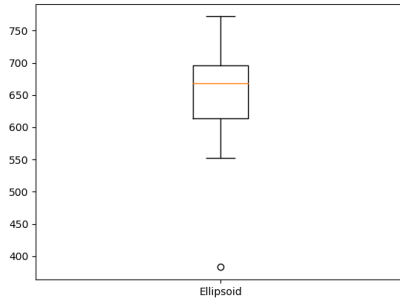


(a) $\text{ACO}_{\mathbb{R}}\text{-simple}$



(b) $\text{ACO}_{\mathbb{R}}\text{-simple}$

Abbildung 14: Vergleich zwischen linearer und Standardabweichung (Ellipsoid)



(a) $\text{ACO}_{\mathbb{R}}\text{-very-simple}$



(b) $\text{ACO}_{\mathbb{R}}\text{-very-simple}$

Abbildung 15: Vergleich zwischen linearer und Standardabweichung (Ellipsoid)

Dabei zeigt der linke Boxplot (a) immer die Anzahl der notwendigen Funktionsauswertungen, mit linearer Abweichung, um das bekannte Ergebnis zu erreichen und der rechte Boxplot (b), die mit Standardabweichung.

Abbildungen 14 und 15 sind auch leider die einzigen Abbildungen, welche als Vergleichsmöglichkeit für lineare und Standardabweichung für Funktionen aus Tabelle ?? herangezogen werden können, da beide Algorithmen, $\text{ACO}_{\mathbb{R}}\text{-simple}$ und $\text{ACO}_{\mathbb{R}}\text{-very-simple}$, für Testfunktionen aus Tabelle ??, außer für „Ellipsoid“, mit der Standardabweichung als Intervallauswertung nicht konvergierte.

Andere Funktionen wurden nach diesen Ergebnisses nicht mehr getestet, da die Funktionen aus Tabelle ?? die einfachen Funktionen aus den verfügbaren Testproblemen darstellen und wenn für diese Funktionen keine Konvergenz zu beobachten ist, dann wird für die anderen erst recht keine zu beobachten sein.

Somit kann geschlussfolgert werden, dass die Intervallberechnung anhand Formel (36), also der linearen Abweichung, die bessere ist.

13. Vergleich von $\text{ACO}_{\mathbb{R}}$, $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple

Wie in der Betrachtung der Einflüsse der Parameter auf $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple sich bereits abgezeichnet hat, gehen wir davon aus, dass $\text{ACO}_{\mathbb{R}}$ -very-simple bei einfachen Testproblemen der bessere Algorithmus ist, da wenn eine Funktion hinreichend einfach ist, es nicht notwendig ist, bereits gefundene gute Lösungen zu einer noch besseren Lösung aufwendig zu verbinden und kann man zu einfacheren Mitteln der Lösungsfindung greifen. Deswegen gehen wir davon aus, dass $\text{ACO}_{\mathbb{R}}$ -very-simple im Vergleich in den Tabellen 12 und 13 als der bessere Algorithmus hervorgehen wird.

In den Tabellen dieses Abschnittes werden ferner als Bestwert immer die Bestwerte aller Algorithmen aus *Socha und Dorigo* [22] betrachtet und zusätzlich der Algorithmen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple, wie sie in den Tabellen aufgeführt werden, weswegen in Tabelle 12 und Tabelle 13 verschiedene Bestwerte zu sehen sind, da in Tabelle 12 $\text{ACO}_{\mathbb{R}}$ -very-simple nicht gelistet ist und somit seine Bestwerte in den Vergleich nicht einfließen.

Tabelle 12: Vergleich der Ergebnisse zwischen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ basierend auf *Socha und Dorigo* [22] und damit auch auf *Kern et al., 2004* [12]

Testfunktion	$\text{ACO}_{\mathbb{R}}$ -simple	$\text{ACO}_{\mathbb{R}}$	Bestwert
Cigar	3.5 (13516)	1.4 (5376)	1.0 (3840)
Diagonal Plane	50.2 (8531)	1.0 (170)	1.0 (170)
Ellipsoid	1.0 (947)	12.2 (11570)	1.0 (947)
Paraboloid	7.3 (10041)	1.1 (1570)	1.0 (1370)
Plane	63.9 (11176)	1.0 (175)	1.0 (175)
Rosenbrock	∞	*1.1 (7909)	1.0 (7190)
Tablet	4.2 (10812)	1.0 (2567)	1.0 (2567)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo ein * steht, wurde die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht, und dort wo ∞ steht wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Wie in Tabelle 12 zu sehen, ist $\text{ACO}_{\mathbb{R}}$ der bessere Algorithmus. $\text{ACO}_{\mathbb{R}}$ -simple mag zwar bei einer Testfunktion vorne liegen, jedoch ist $\text{ACO}_{\mathbb{R}}$ -simple zu schlecht bei den Maximierungsproblemen und auch nicht so gut bei den anderen Testfunktionen, vor allem bei „Rosenbrock“, wo der Algorithmus das Optimum gar nicht findet.

Tabelle 13: Vergleich der Ergebnisse zwischen $\text{ACO}_{\mathbb{R}}$ -very-simple und $\text{ACO}_{\mathbb{R}}$ basierend auf *Socha und Dorigo* [22] und damit auf *Kern et al., 2004* [12]

Testfunktion	$\text{ACO}_{\mathbb{R}}$ -very-simple	$\text{ACO}_{\mathbb{R}}$	Bestwert
Cigar	1.0 (2037)	2.6 (5376)	1.0 (2037)
Diagonal Plane	1.0 (148)	1.1 (170)	1.0 (148)
Ellipsoid	1.0 (644)	18 (11570)	1.0 (644)
Paraboloid	1.1 (1483)	1.1 (1570)	1.0 (1370)
Plane	1.0 (131)	1.3 (175)	1.0 (131)
Rosenbrock	∞	*1.1 (7909)	1.0 (7190)
Tablet	1.0 (1751)	1.5 (2567)	1.0 (1751)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo ein * steht, wurde die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht und dort wo ∞ steht wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Wie in Tabelle 13 zu sehen, ist $\text{ACO}_{\mathbb{R}}$ -very-simple, bei einfachen Testproblemen, der bessere Algorithmus in allen Testproblemen, bis auf „Rosenbrock“. Hätte $\text{ACO}_{\mathbb{R}}$ -very-simple auch bei diesem Problem eine Lösung gefunden, wäre er, für einfache Testprobleme, der klare Favorit gewesen. So aber kann man sagen, dass die Algorithmen, bei einfachen Testproblemen, ebenbürtig sind, besonders, da $\text{ACO}_{\mathbb{R}}$ bei den anderen Testfunktionen nur wenig schlechter ist, außer „Ellipsoid“.

Auch wenn aus den Tabellen 12 und 13 bereits entnommen werden kann, dass von den beiden Algorithmen $\text{ACO}_{\mathbb{R}}$ -simple und $\text{ACO}_{\mathbb{R}}$ -very-simple, $\text{ACO}_{\mathbb{R}}$ -very-simple der bessere ist, betrachten wir in Tabelle 14 nochmal eine direkte Gegenüberstellung, um ein noch klareres Ergebnis zu erhalten.

Tabelle 14: Vergleich der Ergebnisse zwischen $\text{ACO}_{\mathbb{R}}\text{-simple}$ und $\text{ACO}_{\mathbb{R}}\text{-very-simple}$

Testfunktion	$\text{ACO}_{\mathbb{R}}\text{-simple}$	$\text{ACO}_{\mathbb{R}}\text{-very-simple}$
Cigar	6.6 (13516)	1.0 (2037)
Diagonal Plane	57.6 (8531)	1.0 (148)
Ellipsoid	1.5 (947)	1.0 (644)
Paraboloid	6.8 (10041)	1.0 (1483)
Plane	85.3 (11176)	1.0 (131)
Rosenbrock	∞	∞
Tablet	6.2 (10812)	1.0 (1751)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Dort, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Wie in Tabelle 14 nochmal deutlich zu sehen ist, ist $\text{ACO}_{\mathbb{R}}\text{-very-simple}$, bei einfach Testproblemen, der bessere Algorithmus.

Tabelle 15: Zweiter Teil des Vergleiches der Ergebnisse zwischen $\text{ACO}_{\mathbb{R}}\text{-simple}$ und $\text{ACO}_{\mathbb{R}}$ basierend auf *Socha und Dorigo* [22] und damit auf *Kern et al., 2004* [12]

Testfunktion	$\text{ACO}_{\mathbb{R}}\text{-simple}$	$\text{ACO}_{\mathbb{R}}$	Bestwert
B_2	5.3 (2272)	1.3 (544)	1.0 (430)
Branin RCOS	6.5 (1600)	3.5 (858)	1.0 (245)
Easom	-	1.0 [98%] (772)	1.0 (772)
De Jong	2.9 (965)	1.0 ($\sim$ 392)	1.0 (329)
Goldstein and Price	4.0 (917)	1.7 (384)	1.0 (231)
Griewangk	∞	1.0 [61%] (1390)	1.0 (1390)
Hartmann(3,4)	-	1.0 (342)	1.0 (342)
Hartmann(6,4)	2.3 (1714)	1.0 (722)	1.0 (722)
Martin and Gaddy	2.8 (974)	1.0 (345)	1.0 (345)
Paraboloid(6)	3.0 (2308)	1.6 (781)	1.0 (781)
Rosenbrock(2)	15.2 (7317)	1.7 (820)	1.0 (480)
Rosenbrock(5)	∞	1.2 [97%] (2487)	1.0 (2142)
Shekel(4,5)	2.7 (1642)	1.3 [57%] (787)	1.0 [76%] (610)
Shekel(4,7)	2.3 (1595)	1.1 [79%] (748)	1.0 [83%] (680)
Shekel(4,10)	2.4 (1559)	1.1 [81%] (715)	1.0 [83%] (650)
Zakharov(2)	3.4 (664)	1.7 (332)	1.0 (195)
Zakharov(5)	5.7 (4157)	1.0 (727)	1.0 (727)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Bei der Funktion „De Jong“ konnte für $\text{ACO}_{\mathbb{R}}$ die genaue Anzahl an Funktionsauswertungen nicht ermittelt werden.

Wie in Tabelle 15 zu sehen, ist $\text{ACO}_{\mathbb{R}}\text{-simple}$ der schlechtere Algorithmus, er ist sogar bei einer Funktion viel schlechter als $\text{ACO}_{\mathbb{R}}$.

Die Easom-Testfunktion konnte leider nicht angegeben werden, da bei dem Initialisierungsintervall zu große Werte für den Algorithmus auftreten und es zu einem Überlauf kommt. Bei kleineren Initialisierungsintervallen konvergiert der Algorithmus jedoch schnell.

Die Hartmann(3,4)-Testfunktion konnte leider nicht angegeben werden, da die Anzahl der durchschnittlichen Funktionsauswertungen bei dieser Testfunktion bei 4 lag und es ist eher wahrscheinlich, dass bei dem Auswerten, Berechnen oder Initialisieren ein Fehler

aufgetreten ist, als dass der Algorithmus wirklich nur 4 Funktionsauswertungen braucht.

Auf der anderen Seite, wenn die Genauigkeit auf $|f - f^*| < |\epsilon_1 f + \epsilon_2|$, $\epsilon_1 = \epsilon_2 = 10^{-10}$ angehoben wird, braucht der Algorithmus mehrere Tausend Funktionsauswertungen (5000-12000) und findet das Optimum sogar manchmal nicht, was viel eher dafür spricht, dass es hauptsächlich an der Initialisierung gepaart mit der geringen Anforderung an die Genauigkeit liegt und es kein Fehler der Berechnung ist.

Weiter haben Tests ergeben, dass selbst bei einer Archivgröße von $k=10$, statt $k=100$, das selbe Ergebnis beobachtet werden kann, was die Initialisierung ausschließt, da 10 zufällige Lösungen, selbst wenn es gute sind, den Effekt nicht erklären können und vor allem nicht bei allen 50 Durchläufen, da auch mal schlechte Lösungen ins Archiv hätten aufgenommen werden müssen. Das Ergebnis konnte jedoch konstant beobachtet werden, unabhängig von der Archivgröße.

Damit lässt sich die geringe Anforderung an die Genauigkeit als Quelle des Effektes feststellen. Auf der anderen Seite ist das Ergebnis einfach zu gut, um lediglich mit einer geringen Anforderung an die Genauigkeit erklärt werden zu können. Ich gehe davon aus, dass es noch andere Faktoren gibt und führe das Ergebnis in der Tabelle 15 nicht auf.

Tabelle 16: Zweiter Teil des Vergleiches der Ergebnisse zwischen $\text{ACO}_{\mathbb{R}}$ -very-simple und $\text{ACO}_{\mathbb{R}}$ basierend auf *Socha und Dorigo* [22] und damit auf *Kern et al., 2004* [12]

Testfunktion	$\text{ACO}_{\mathbb{R}}$ -very-simple	$\text{ACO}_{\mathbb{R}}$	Bestwert
B_2	1.7 (747)	1.3 (544)	1.0 (430)
Branin RCOS	3.0 (747)	3.5 (858)	1.0 (245)
Easom	-	1.0 [98%] (772)	1.0 (772)
De Jong	1.0 (319)	1.2 ($\sim$ 392)	1.0 (319)
Goldstein and Price	1.5 (344)	1.7 (384)	1.0 (231)
Griewangk	12.3 [44%] (17062)	1.0 [61%] (1390)	1.0 (1390)
Hartmann(3,4)	-	1.0 (342)	1.0 (342)
Hartmann(6,4)	1.0 (377)	1.9 (722)	1.0 (377)
Martin and Gaddy	1.0 (356)	1.0 (345)	1.0 (345)
Paraboloid(6)	1.0 (490)	1.6 (781)	1.0 (490)
Rosenbrock(2)	2.5 (1195)	1.7 (820)	1.0 (480)
Rosenbrock(5)	18.4 [54%] (39345)	1.2 [97%] (2487)	1.0 (2142)
Shekel(4,5)	1.0 (445)	1.8 [57%] (787)	1.0 (445)
Shekel(4,7)	1.0 (444)	1.8 [79%] (748)	1.0 (444)
Shekel(4,10)	1.0 (443)	1.6 [81%] (715)	1.0 (443)
Zakharov(2)	1.5 (293)	1.7 (332)	1.0 (195)
Zakharov(5)	1.0 (694)	1.7 (727)	1.0 (694)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Bei der Funktion „De Jong“ konnte für $\text{ACO}_{\mathbb{R}}$ die genaue Anzahl an Funktionsauswertungen nicht ermittelt werden.

Wie in Tabelle 16 zu sehen, ist es nicht mehr klar, welcher der Algorithmen vorzuziehen ist. $\text{ACO}_{\mathbb{R}}$ ist bei vier Testfunktionen der beste Algorithmus, während $\text{ACO}_{\mathbb{R}}$ -very-simple bei Sieben. Beide Algorithmen sind jedoch bei einem Teil der Testfunktionen schlechter als das Optimum.

Die Easom-Testfunktion konnte leider nicht angegeben werden, da bei den Initialisierungsintervall zu große Werte für den Algorithmus auftreten und es zu einem Überlauf kommt. Bei kleineren Initialisierungsintervallen konvergiert der Algorithmus jedoch sehr schnell.

Die Hartmann(3,4) Testfunktion konnte leider nicht angegeben werden, da die Anzahl

der durchschnittlichen Funktionsauswertungen bei dieser Testfunktion bei 2 lag und es ist eher wahrscheinlich, dass bei dem Auswerten, Berechnen oder Instanzieren ein Fehler aufgetreten ist, als dass wirklich nur 2 Funktionsauswertungen nötig waren. Der Fehler konnte leider nicht gefunden werden.

Auf der anderen Seite, wenn die Genauigkeit auf $|f - f^*| < |\epsilon_1 f + \epsilon_2|$, $\epsilon_1 = \epsilon_2 = 10^{-10}$ angehoben wird, braucht der Algorithmus auch hier mehrere Tausend Funktionsauswertungen (1000-5000), was viel eher dafür spricht, dass es hier genauso hauptsächlich an der Initialisierung gepaart mit der geringen Anforderung an die Genauigkeit liegt und es kein Fehler der Berechnung ist.

Hier ist die Argumentation die selbe, nämlich dass es an der geringen Genauigkeit liegt, dass so ein Ergebnis beobachtet werden kann.

Allerdings entscheide ich mich genauso wie in Tabelle 15 das Ergebnis in der Tabelle 16 nicht aufzuführen.

Wie bereits erwähnt, geht es über den Umfang dieser Arbeit hinaus, $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ mit anderen Algorithmen aus *Socha und Dorigo* [22] als $\text{ACO}_{\mathbb{R}}$ zu vergleichen, weswegen nur betrachtet wird, ob einer der beiden Algorithmen der beste ist oder nicht, wie in Tabelle 13 und Tabelle 16 verdeutlicht.

Deswegen lässt es sich, durch diese eingeschränkte Sichtweise, leicht urteilen, dass $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ der Favorit ist. Er mag $\text{ACO}_{\mathbb{R}}$ zwar bei einer einfacheren Testfunktion (Rosenbrock) unterlegen sein, wie in Tabelle 13 zu sehen, ist jedoch auch im Schnitt besser, wenn es um schwerere Testfunktionen geht, wie in Tabelle 16 klar wird. $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ mag bei zwei der schwereren Testfunktionen (Griewangk und Rosenbrock(5)) wesentlich mehr Funktionsauswertungen brauchen als $\text{ACO}_{\mathbb{R}}$, ist jedoch bei den besonders schweren Testfunktionen, den Hartmann- und Shekelfunktionen, $\text{ACO}_{\mathbb{R}}$ überlegen.

Wie auch bei den einfachen Testfunktionen, um das Ergebnis klarer zu sehen, vergleichen wir $\text{ACO}_{\mathbb{R}}\text{-simple}$ und $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ direkt in Tabelle 17

Tabelle 17: Zweiter Teil des Vergleiches der Ergebnisse zwischen $\text{ACO}_{\mathbb{R}}\text{-simple}$ und $\text{ACO}_{\mathbb{R}}\text{-very-simple}$

Testfunktion	$\text{ACO}_{\mathbb{R}}\text{-simple}$	$\text{ACO}_{\mathbb{R}}\text{-very-simple}$
B_2	3.0 (2272)	1.0 (747)
Branin RCOS	2.1 (1600)	<i>1.0</i> (747)
Easom	-	-
De Jong	3.0 (965)	1.0 (319)
Goldstein and Price	2.7 (917)	1.0 (344)
Griewangk	∞	1.0 [44%] (17062)
Hartmann(3,4)	-	-
Hartmann(6,4)	5.0 (1714)	1.0 (377)
Martin and Gaddy	2.6 (974)	1.0 (356)
Paraboloid(6)	4.7 (2308)	1.0 (490)
Rosenbrock(2)	6.2 (7317)	1.0 (1195)
Rosenbrock(5)	∞	1.0 [54%] (39345)
Shekel(4,5)	3.7 (1642)	1.0 (445)
Shekel(4,7)	3.6 (1595)	1.0 (444)
Shekel(4,10)	3.5 (1559)	1.0 (443)
Zakharov(2)	2.3 (664)	1.0 (293)
Zakharov(5)	6.0 (4157)	1.0 (694)

Dargestellt ist zuerst die relative Anzahl an Funktionsauswertungen und in Klammern danach die absolute. Bei Ergebnissen, wo die erforderliche Genauigkeit nicht bei jedem Durchlauf des Algorithmus erreicht wurde, ist der Anteil der erfolgreichen Durchläufe in Prozent in eckigen Klammern angegeben, wo ∞ steht, wurde die Genauigkeit bei keinem Durchlauf des Algorithmus erreicht.

Wie in Tabelle 17 nochmal deutlich zu sehen, ist $\text{ACO}_{\mathbb{R}}\text{-very-simple}$, auch bei schwereren Testproblemen, der bessere Algorithmus.

14. Konvergenzbetrachtung

Zwar ist Tabelle 16 bereits sehr aussagekräftig, aber bei einem Urteil, dass sich ausschließlich auf diese Tabelle stützt, geht verloren, dass die Leistung, die $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ liefert, schon ausreichend sein kann, selbst wenn der Algorithmus nicht konvergiert, gemessen daran, wie einfach, somit schnell der Algorithmus ist, im Vergleich zu $\text{ACO}_{\mathbb{R}}$. Es liegen zwar keine Daten zur Zeitkomplexität von $\text{ACO}_{\mathbb{R}}$ vor, aber wir können davon ausgehen, dass $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ schneller ist.

Deswegen betrachten wir in diesem Abschnitt, ob $ACO_{\mathbb{R}}$ -very-simple gegen Werte konvergiert, die den bekannten Optimalwerten der Testprobleme nahe sind.

Es mag zwar stimmen, dass bei Algorithmen zur Lösung stetiger Probleme, es mehr Sinn macht, die Anzahl der Funktionsauswertungen als Maßstab der Güte eines Verfahrens zu nehmen, jedoch ist der in *Socha und Dorigo* [22] beschriebene Algorithmus selbst von keiner kleinen Zeitkomplexität. Weswegen das dort gebrachte Argument, die Laufzeit zum Vergleich nicht hinzuziehen, an Stärke verliert.

Es ist immer noch sehr stark, da alle dafür angebrachte Argumente wahr und valide sind, wie dass die Laufzeit von der verwendeten Programmiersprache abhängt, vom Compiler oder von dem Rechner und somit man zwar die Laufzeiten messen könnte aber diese weniger Aussagekräftig ist als die Anzahl der Funktionsauswertungen aber die Verfasser der Quelle hätten die Laufzeit trotzdem mit angeben können, da es bei dem Vergleich mit den dieser Arbeit beiliegenden Verfahren sehr interessant wäre zu sehen, wie viel schneller die hier vorgestellten Algorithmen sind und ob ihre Geschwindigkeit ein Argument für ihre Verwendung sein könnte.

Deswegen wäre an dieser Stelle die Zeitkomplexität von $ACO_{\mathbb{R}}$ besonders interessant gewesen, da, wenn der Algorithmus $ACO_{\mathbb{R}}$ doch noch langsamer ist als ohnehin erwartet, die Wahl zwischen den Algorithmen, noch mehr in Richtung $ACO_{\mathbb{R}}$ -very-simple ausfallen würde. Betrachten wir nämlich den Fall, wo viele Arbeitsschritte mit geringerer Präzision nacheinander schnell abgearbeitet werden müssen und $ACO_{\mathbb{R}}$ noch langsamer als erwartet ist, wäre $ACO_{\mathbb{R}}$ -very-simple, $ACO_{\mathbb{R}}$ tatsächlich eindeutig vorzuziehen, da in den folgenden Abschnitten klar zu sehen sein wird, dass $ACO_{\mathbb{R}}$ -very-simple dem geforderten Optimalwert des Testproblems, wo das Verfahren keine Lösung findet, hinreichend nahe kommt.

Somit werden wir am Ende der Konvergenzbetrachtung festhalten können, $ACO_{\mathbb{R}}$ -very-simple ist, bei geringerer Anforderung an die Genauigkeit der Lösung, $ACO_{\mathbb{R}}$ noch mehr als sonst vorzuziehen, sofern $ACO_{\mathbb{R}}$ langsam genug ist.

14.1. Rosenbrock

Die Rosenbrockfunktion nimmt ihren Optimalwert $y=0$ an bei $\vec{x} = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]$.

Der durchschnittlich berechnete Funktionswert bei 10000 Funktionsauswertungen war ungefähr $y=13$, mit $x_i \in [-1, 1] \forall i \in \{1, 2, 3, \dots, 10\}$

Wie zu sehen, ist der Funktionswert etwas weiter entfernt von dem Optimum aber $\vec{x}$ ist nah dran am Argument des Optimums.

Somit können wir schlussfolgern, dass $ACO_{\mathbb{R}}$ -very-simple hinreichend gut ist, auch bei der Funktion, wo der Algorithmus keine Lösung findet.

15. Kurzzusammenfassung

Es ist erstaunlich, wie die Vereinfachung abschneidet. Zwar mag $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ nicht immer der beste Algorithmus sein, aber er ist der beste bei 12 von 24 Testfunktion, also bei der Hälfte aller Testfunktion. Damit ist er $\text{ACO}_{\mathbb{R}}$ überlegen. Natürlich sind fünf dieser Testfunktionen einfach, aber selbst bei den schweren Funktionen weiß $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ zu überzeugen, insbesondere, da er sehr schnell ist und unter bestimmten Umständen sogar auf ein exaktes Ergebnis verzichten kann.

Insbesondere ist es bemerkenswert, dass es der Algorithmus $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ ist, der sich mit $\text{ACO}_{\mathbb{R}}$ messen kann und nicht $\text{ACO}_{\mathbb{R}}\text{-simple}$, obwohl unsere Intuition hier gerade das gegenteilige Ergebnis vorhergesagt hat.

Abschließend lässt sich sagen, dass durch $\text{ACO}_{\mathbb{R}}\text{-very-simple}$ ein weiterer ausgezeichneter Algorithmus der Klasse der Ameisenalgorithmen beigetreten ist, dazu noch einer, der so einfach ist, wie es nur geht und damit die Frage aus der Einleitung beantwortet. Es geht viel einfacher als bisher – eben very simple.

Ferner ist das dieser Arbeit beiliegende Programm so geschrieben, dass es sich leicht um weitere Suchverfahren und Testfunktionen erweitern lässt.

Es ist so geschrieben, dass nur das Verfahren selbst programmiert und der Rest so beibehalten werden kann – das heißt die Testfunktionen können für das neue Verfahren weiter verwendet werden, ohne weitere Änderung am Code, genauso wie die Funktionen zum generieren der Boxplots. Ebenso einfach können neuen Testfunktionen eingebunden werden. Es ist lediglich notwendig die Testfunktion hinzuzufügen.

Das macht das Ausprobieren neuer Suchalgorithmen bedeutend einfacher und schneller. Zur Anschauung, wie einfach, wurde in das dieser Arbeit beiliegende Programm der Standard-PSO Algorithmus eingebaut.

Literatur

- [1] BILCHEV, G. ; PARMEE, I. C.: The ant colony metaphor for searching continuous design spaces. In: *AISB workshop on evolutionary computing* Springer, 1995, S. 25–39
- [2] BLUM, C. : Ant colony optimization: Introduction and recent trends. In: *Physics of Life reviews* 2 (2005), Nr. 4, S. 353–373
- [3] BONABEAU, E. ; DORIGO, M. ; THERAULAZ, G. : *Swarm intelligence: from natural to artificial systems*. Oxford university press, 1999 (1)
- [4] CAMAZINE, S. ; DENEUBOURG, J. ; FRANKS, N. ; SNEYD, J. ; THERAULAZ, G. u. a.: *Self-organization in Biological Systems 2003 Princeton*. 2003
- [5] CHEN, W.-N. ; ZHANG, J. : A set-based discrete PSO for cloud workflow scheduling with user-defined QoS constraints. In: *2012 IEEE International Conference on Systems, Man, and Cybernetics (SMC)* IEEE, 2012, S. 773–778
- [6] CHRISTIAN, B. ; DANIEL, M. : Swarm intelligence introduction and application. In: *Natural Computing Series, Springer* (2008), S. 87–100
- [7] DORIGO, M. ; MANIEZZO, V. ; COLORNI, A. : Ant system: optimization by a colony of cooperating agents. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 26 (1996), Nr. 1, S. 29–41
- [8] DORIGO, M. ; STÜTZLE, T. : Ant colony optimization: overview and recent advances. In: *Handbook of metaheuristics* (2018), S. 311–351
- [9] ENGELBRECHT, A. P.: Fundamentals of Computational Swarm Intelligence. In: *Hoboken, NJ: Wiley* (2005)
- [10] GUNTSCH, M. ; MIDDENDORF, M. : A Population Based Approach for ACO. In: *Workshops on Applications of Evolutionary Computing*, Springer Berlin Heidelberg, 2002, S. 72–81
- [11] GUO, C.-x. ; HU, J.-s. ; YE, B. ; CAO, Y.-j. : Swarm intelligence for mixed-variable design optimization. In: *Journal of Zhejiang University-SCIENCE A* 5 (2004), Nr. 7, S. 851–860
- [12] KERN, S. ; MÜLLER, S. D. ; HANSEN, N. ; BÜCHE, D. ; OCENASEK, J. ; KOU-MOUTSAKOS, P. : Learning probability distributions in continuous evolutionary algorithms—a comparative review. In: *Natural Computing* 3 (2004), Nr. 1, S. 77–112
- [13] KUPFER, E. ; LE, H. T. ; ZITT, J. ; LIN, Y.-C. ; MIDDENDORF, M. : A Hierarchical Simple Probabilistic Population-Based Algorithm Applied to the Dynamic TSP. In: *2021 IEEE Symposium Series on Computational Intelligence (SSCI)* IEEE, 2021, S. 1–8

- [14] LIAO, T. ; SOCHA, K. ; OCA, M. A. M. ; STÜTZLE, T. ; DORIGO, M. : Ant colony optimization for mixed-variable optimization problems. In: *IEEE Transactions on evolutionary computation* 18 (2013), Nr. 4, S. 503–518
- [15] LIN, Y.-C. ; CLAUSS, M. ; MIDDENDORF, M. : Simple probabilistic population-based optimization. In: *IEEE Transactions on Evolutionary Computation* 20 (2015), Nr. 2, S. 245–262
- [16] MADORSKIY, F. C.: *Vergleich zweier Varianten der Lösungserzeugung von Ameisenalgorithmen zur Funktionsoptimierung*, Universität Leipzig, Bachelorarbeit, 2002
- [17] MERKLE, D. ; MIDDENDORF, M. : On the behavior of ACO algorithms: Studies on simple problems. In: *Metaheuristics: computer decision-making*. Springer, 2003, S. 465–480
- [18] MOLGA, M. ; SMUTNICKI, C. : Test functions for optimization needs. In: *Test functions for optimization needs* 101 (2005)
- [19] POHLHEIM, H. : Examples of objective functions. In: *Retrieved* 4 (2007), Nr. 10, S. 2012
- [20] SANDGREN, E. : Nonlinear integer and discrete programming in mechanical design optimization. (1990), S. 223–229
- [21] SOCHA, K. : *Ant colony optimisation for continuous and mixed-variable domains*. VDM Publishing Saarbrücken, 2009
- [22] SOCHA, K. ; DORIGO, M. : Ant colony optimization for continuous domains. In: *European journal of operational research* 185 (2008), Nr. 3, S. 1155–1173
- [23] STÜTZLE, T. ; DORIGO, M. u. a.: ACO algorithms for the traveling salesman problem. In: *Evolutionary algorithms in engineering and computer science* 4 (1999), S. 163–183
- [24] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/hart3.html> (abgerufen am 17.08.2019)
- [25] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/hart6.html> (abgerufen am 17.08.2019)
- [26] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/shekel.html> (abgerufen am 17.08.2019)
- [27] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: <https://www.sfu.ca/~ssurjano/branin.html> (abgerufen am 19.08.2019)

- [28] SURJANOVIC, S. ; BINGHAM, D. : Virtual Library of Simulation Experiments: Test Function and Datasets. In: *<https://www.sfu.ca/~ssurjano/goldpr.html>* (abgerufen am 19.08.2019)

Tabellenverzeichnis

1.	Parametereinstellungen ACO _R -simple	13
2.	Parametereinstellungen ACO _R -very-simple	14
3.	Testfunktionen zum Vergleich mit Algorithmen, die probabilistisches Lernen einsetzen	24
4.	erster Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Probleme adaptiert wurden	25
5.	zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Probleme adaptiert wurden	26
6.	zweiter Teil der Testfunktionen zum Vergleich mit anderen mit Ameisenalgorithmen verwandten Methoden und Metaheuristiken, die für stetige Probleme adaptiert wurden	27
7.	Standarddrahtdurchmesser für die Schraubenfeder angegeben in Zoll (inch)	33
8.	Intervallgrenzen	33
9.	Definitionsbereiche	34
10.	Constraints	34
11.	Gewichte	35
12.	Vergleich der Ergebnisse zwischen ACO _R -simple und ACO _R basierend auf <i>Socha und Dorigo</i> [22] und damit auch auf <i>Kern et al., 2004</i> [12]	43
13.	Vergleich der Ergebnisse zwischen ACO _R -very-simple und ACO _R basierend auf <i>Socha und Dorigo</i> [22] und damit auf <i>Kern et al., 2004</i> [12]	44
14.	Vergleich der Ergebnisse zwischen ACO _R -simple und ACO _R -very-simple	45
15.	Zweiter Teil des Vergleiches der Ergebnisse zwischen ACO _R -simple und ACO _R basierend auf <i>Socha und Dorigo</i> [22] und damit auf <i>Kern et al., 2004</i> [12]	46
16.	Zweiter Teil des Vergleiches der Ergebnisse zwischen ACO _R -very-simple und ACO _R basierend auf <i>Socha und Dorigo</i> [22] und damit auf <i>Kern et al., 2004</i> [12]	48
17.	Zweiter Teil des Vergleiches der Ergebnisse zwischen ACO _R -simple und ACO _R -very-simple	50

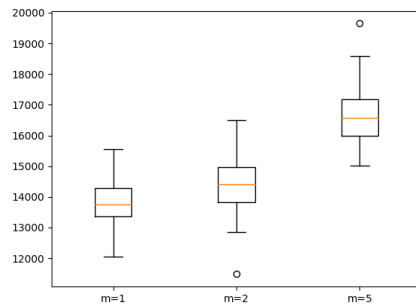
Abbildungsverzeichnis

1.	Experimentaufstellung in Anlehnung an <i>Engelbrecht</i> [9]	3
2.	Paraboloid - Visualisierung in 3D	28
3.	Easom-Testfunktion: Vergleich zwischen gesamtem Suchraum (links) und Umgebung um das Optimum (rechts)	28
4.	Verschiedene Auflösungen der Griewangk-Testfunktion	29
5.	Rosenbrock - Visualisierung in 3D	30
6.	Hartmann-Testfunktionen: Hartmann(2,4) (links) und Hartmann(2,6) (rechts)	30

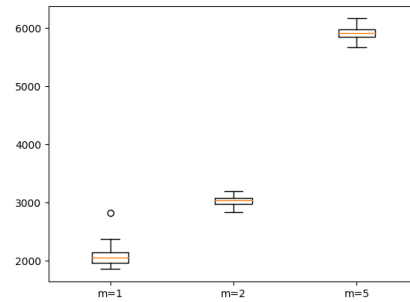
7.	Shekel-Testfunktionen: Shekel(2,5) (links), Shekel(2,7) (mittig) und Shekel(2,10) (rechts)	31
8.	gefilterter und ungefilterter Suchraum für das CSD-Problem	36
9.	gefilterter und ungefilterter Raum	36
10.	Schematische Zeichnung des TISD, wie dargestellt in [14]	37
11.	Ellipsoid Vergleich m	38
12.	Ellipsoid Vergleich q	39
13.	Ellipsoid Vergleich ξ	40
14.	Vergleich zwischen linearer und Standardabweichung (Ellipsoid)	42
15.	Vergleich zwischen linearer und Standardabweichung (Ellipsoid)	42
16.	Cigar Vergleich m	I
17.	Paraboloid Vergleich m	I
18.	Tablet Vergleich m	II
19.	Cigar Vergleich q	II
20.	Paraboloid Vergleich q	III
21.	Tablet Vergleich q	III
22.	Cigar Vergleich ξ	IV
23.	Paraboloid Vergleich ξ	IV
24.	Tablet Vergleich ξ	IV

Anlagen

A. Einfluss der Anzahl der Ameisen

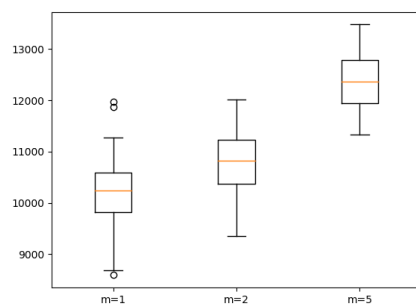


(a) ACO_R-simple

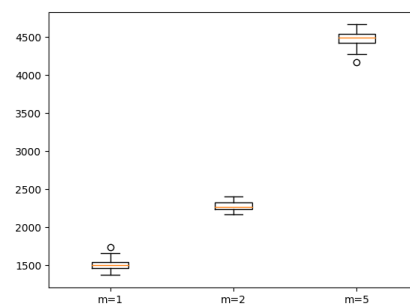


(b) ACO_R-very-simple

Abbildung 16: Cigar Vergleich m



(a) ACO_R-simple



(b) ACO_R-very-simple

Abbildung 17: Paraboloid Vergleich m

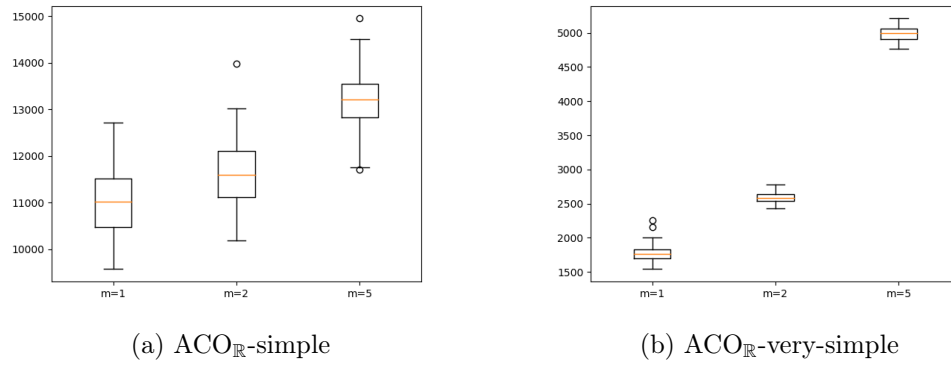


Abbildung 18: Tablet Vergleich m

B. Einfluss des Parameters q

Da bei $ACO_{\mathbb{R}}\text{-very-simple}$ der Parameter q entfällt, beziehen sich alle Abbildungen in diesem Abschnitt nur auf $ACO_{\mathbb{R}}\text{-simple}$.

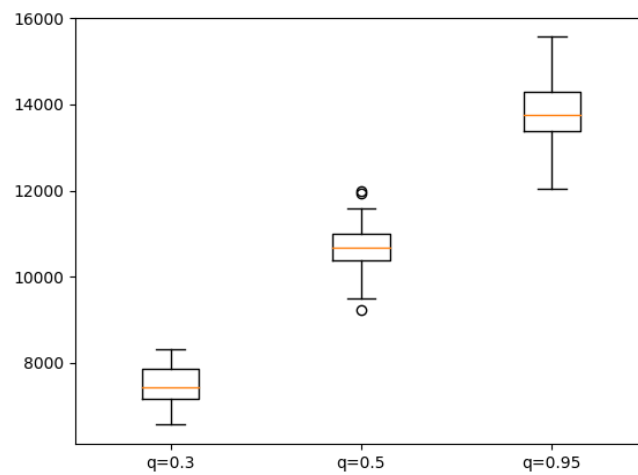


Abbildung 19: Cigar Vergleich q

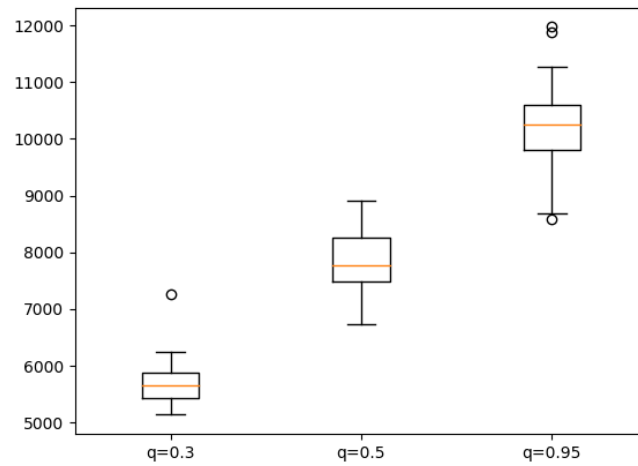


Abbildung 20: Paraboloid Vergleich q

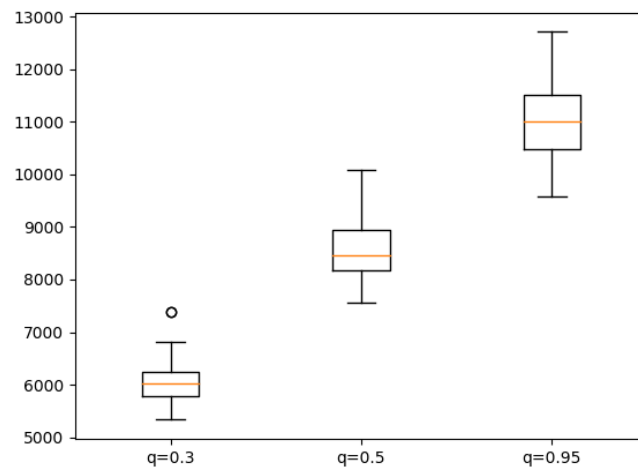
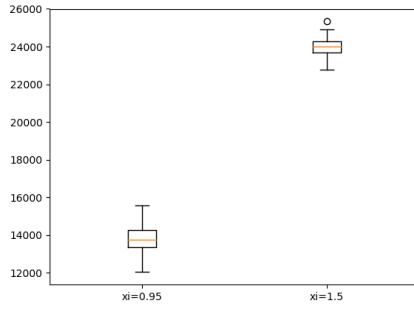


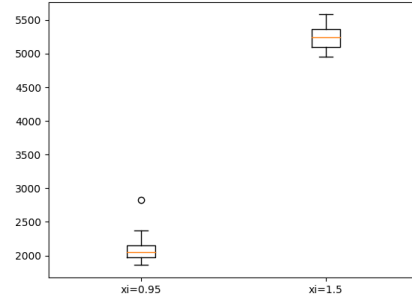
Abbildung 21: Tablet Vergleich q

C. Einfluss des Parameters ξ

Die in diesem Abschnitt enthaltenen Abbildungen haben immer nur zwei Box Plots, da beide Algorithmen für die beiden anderen Testwerte $\xi=0.3$ und $\xi=0.5$ nicht konvergierten.

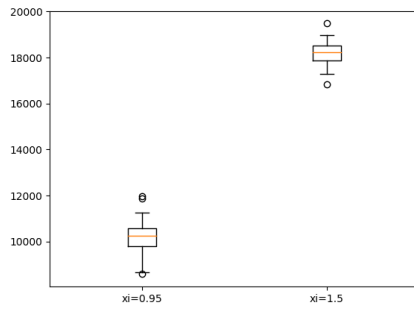


(a) $\text{ACO}_{\mathbb{R}}\text{-simple}$

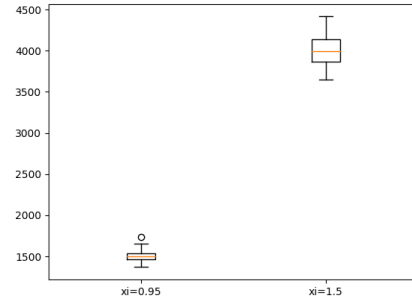


(b) $\text{ACO}_{\mathbb{R}}\text{-very-simple}$

Abbildung 22: Cigar Vergleich xi

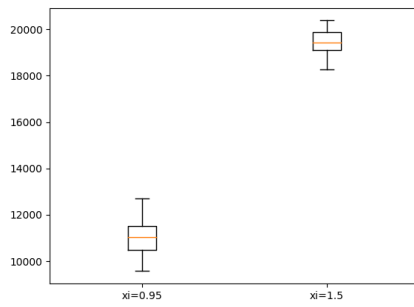


(a) $\text{ACO}_{\mathbb{R}}\text{-simple}$

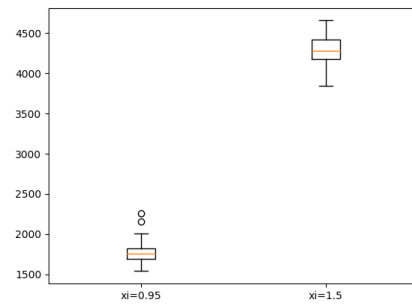


(b) $\text{ACO}_{\mathbb{R}}\text{-very-simple}$

Abbildung 23: Paraboloid Vergleich xi



(a) $\text{ACO}_{\mathbb{R}}\text{-simple}$



(b) $\text{ACO}_{\mathbb{R}}\text{-very-simple}$

Abbildung 24: Tablet Vergleich xi

Erklärung

Ich versichere, dass ich die vorliegende Arbeit selbstständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt habe, insbesondere sind wörtliche oder sinngemäße Zitate als solche gekennzeichnet. Mir ist bekannt, dass Zuwiderhandlung auch nachträglich zur Aberkennung des Abschlusses führen kann. Ich versichere, dass das elektronische Exemplar mit den gedruckten Exemplaren übereinstimmt.

Ort: Leipzig

Datum:

Unterschrift: